

Community-Hangar installieren

Schritt für Schritt mit Bildern: eine eigene Instanz auf einem eigenen Server aufbauen, mit Discord-Anmeldung, Onboarding-Bot und dem ersten RSI-Sync. Du brauchst keine Programmierkenntnisse, aber du solltest auf einem Linux-Server Befehle eingeben können. Zeitaufwand: etwa 45 Minuten.

Stand: 06.10.2026 · Die Bilder stammen von einer echten Neuinstallation (Orga „SpaceTrotter“, Adresse `expg.mcdance.de`). Persönliche Angaben, Kennungen und Geheimnisse sind geschwärzt. Überall, wo `expg.mcdance.de` steht, setzt du deine eigene Adresse ein.

1. Was du vorher brauchst
2. Discord-Anwendung anlegen
3. Dateien auf den Server legen
4. Einstellungen eintragen (.env)
5. HTTPS einrichten (Reverse-Proxy)
6. Starten und prüfen
7. Endpunkt bei Discord eintragen
8. Befehle registrieren
9. Bot auf den Server einladen
10. Orga einrichten
11. Anmelden
12. Ersten RSI-Sync machen
13. Betrieb und Updates
14. Wenn etwas nicht klappt

1 Was du vorher brauchst

Was	Hinweis
Ein Linux-Server oder eine VM (auch ein Container)	Mindestens 1 GB RAM und 5 GB Platz. Docker muss laufen (Version 24 oder neuer, mit dem Befehl <code>docker compose</code>).
Eine Domain oder Subdomain	Zum Beispiel <code>hangar.example.de</code> . Sie muss per DNS auf deinen Server bzw. deinen Reverse-Proxy zeigen. Discord verlangt HTTPS.
Ein Discord-Konto mit Admin-Rechten	Auf dem Server, der die Orga werden soll („Administrator“ oder „Server verwalten“).
Zugriff auf das Discord Developer Portal	Kostenlos, mit demselben Konto: <code>https://discord.com/developers/applications</code>

Nichts davon kostet etwas. Das Docker-Image ist öffentlich und kommt von `ghcr.io/mackedanz/community-hangar-1`.

Deutsches oder englisches Portal. Das Developer Portal ist je nach Spracheinstellung deutsch oder englisch. Die Namen in diesem Heft sind deutsch, die englischen stehen daneben:

Deutsch	Englisch
App erstellen	New Application
Allgemeine Informationen	General Information
Anwendungs-ID · Öffentlicher Schlüssel	Application ID · Public Key
Geheimen Schlüssel zurücksetzen	Reset Secret
Weiterleitungen	Redirects
Token zurücksetzen · Öffentlicher Bot	Reset Token · Public Bot
Interaktions-Endpunkt-URL	Interactions Endpoint URL
Installations-Link · Gildeninstallation	Install Link · Guild Install

Merke dir in jedem Schritt: Auf den Seiten des Developer Portals erscheint nach jeder Änderung unten eine Leiste „Achtung, du hast nicht gespeicherte Änderungen!“. Erst nach einem Klick auf „**Änderungen speichern**“ gilt die Änderung.

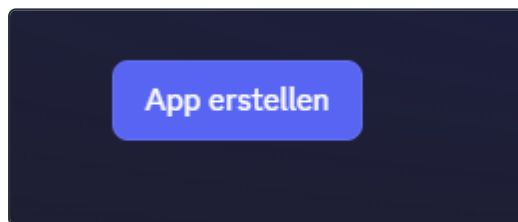
2 Discord-Anwendung anlegen

Gehe auf `https://discord.com/developers/applications` und melde dich an. Während dieses Teils sammelst du vier Werte in einer Textdatei auf deinem Rechner. Du trägst sie später in die `.env` auf dem Server ein.

Wert	Woher	Geheim?
<code>AUTH_DISCORD_ID</code>	Allgemeine Informationen → Anwendungs-ID	nein
<code>DISCORD_PUBLIC_KEY</code>	Allgemeine Informationen → Öffentlicher Schlüssel	nein
<code>AUTH_DISCORD_SECRET</code>	OAuth2 → Geheimer Client-Schlüssel	ja
<code>DISCORD_BOT_TOKEN</code>	Bot → Token	ja

Geheime Werte gehören nur in die `.env` auf dem Server. Nicht in Chats, E-Mails, Bilder oder ein Repository.

2.1 Anwendung erstellen



Im Developer Portal auf „App erstellen“ klicken.

Trage einen Namen ein, zum Beispiel **Community-Hangar**. Der Name erscheint später als Name des Bots und bei der Anmeldung. Setze den Haken bei den Nutzungsbedingungen (das bestätigst du selbst) und klicke auf **Erstellen**.

A dark-themed dialog box titled 'Erstelle eine neue Anwendung' with a close button (X) in the top right. It contains a 'Name' input field with a red asterisk. Below the field is a checkbox with the text: 'Durch Klicken auf Erstellen stimmst du den Nutzungsbedingungen für Entwickler und der Entwicklerrichtlinie von Discord zu.' At the bottom are two buttons: 'Abbrechen' and 'Erstellen'.

Name eintragen, Bedingungen bestätigen, „Erstellen“.

2.2 Anwendungs-ID und Öffentlichen Schlüssel kopieren

Es öffnet sich die Seite **Allgemeine Informationen**. Kopiere mit den Knöpfen **Kopieren** die **Anwendungs-ID** und den **Öffentlichen Schlüssel** in deine Textdatei.

Das Feld „Interaktions-Endpunkt-URL“ bleibt noch leer, es kommt erst in Schritt 7 dran.

Allgemeine Informationen

Wie sollen wir deine Schöpfung nennen? Was hat sie Tolles zu bieten? Welches Icon soll sie auf Discord repräsentieren? Sag es uns hier!

App-Icon

Abmessungen: 1024x1024
Seitenverhältnis: 1:1
Dateitypen: PNG, GIF, JPG, WEBP
Maximale Größe: 10MB

Name

Beschreibung (maximal 400 Zeichen)

Deine Beschreibung wird im Abschnitt „Über mich“ im Profil deines Bots angezeigt.

Tags (maximal 5)

Füge bis zu 5 Tags hinzu, um den Inhalt und die Funktionalität deiner Anwendung zu beschreiben.

Anwendungs-ID

Kopieren

Öffentlicher Schlüssel

Kopieren

Installationsanzahl

Dies ist eine ungefähre Anzahl an Servern und Nutzern, die deine Anwendung installiert haben. Diese Zahlen werden täglich aktualisiert.

0 Server
0 Einzelne Benutzer

Anzahl der Autorisierungen

Dies ist eine angenäherte Anzahl von Nutzern, die eine OAuth2-Autorisierung für deine Anwendung haben. Diese Zahl wird täglich aktualisiert.

0 Einzelne Benutzer

Interaktions-Endpunkt-URL

Du kannst optional einen Interaktions-Endpunkt konfigurieren, um Interaktionen über HTTP-POSTs zu erhalten, anstatt via Gateway mit einem Bot-Benutzer.

Verknüpfte Rollen-Verifizierungs-URL

Du kannst eine Verifizierungs-URL konfigurieren, um deine Anwendung als Anforderung in den Links-Einstellungen einer Serverrolle zu aktivieren

Allgemeine Informationen: Anwendungs-ID und Öffentlicher Schlüssel (hier geschwärzt).

2.3 OAuth2: Geheimnis und Redirect

Öffne im Menü links OAuth2.

Menü links: OAuth2.

+ Erstellen Docs

OAuth2

Nutze Discord als Autorisierungssystem oder verwende unsere API im Namen deiner Nutzer. Füge einen Weiterleitungs-URI hinzu, wähle deine Anwendungsbereiche aus, würfle einen D20 auf Glück, und los!

[Mehr über OAuth2 erfahren](#)

Client-Informationen

Client-ID

Geheimer Client-Schlüssel

Versteckt aus Sicherheitsgründen

Geheimen Schlüssel zurücksetzen

Öffentlicher Client

Öffentliche Clients können die Vertraulichkeit ihrer Client-Anmeldeinformationen nicht wahren (z. B. bei Desktop-/Mobilgeräten, die keinen Server verwenden, um Anfragen zu erstellen)

Weiterleitungen

Du musst mindestens einen URI angeben, damit die Authentifizierung funktioniert. Wenn du einen URI in einer OAuth-Anfrage übermittelst, muss er genau mit einem der URIs übereinstimmen, die du hier eingibst.

Redirect hinzufügen

OAuth2 URL-Generator

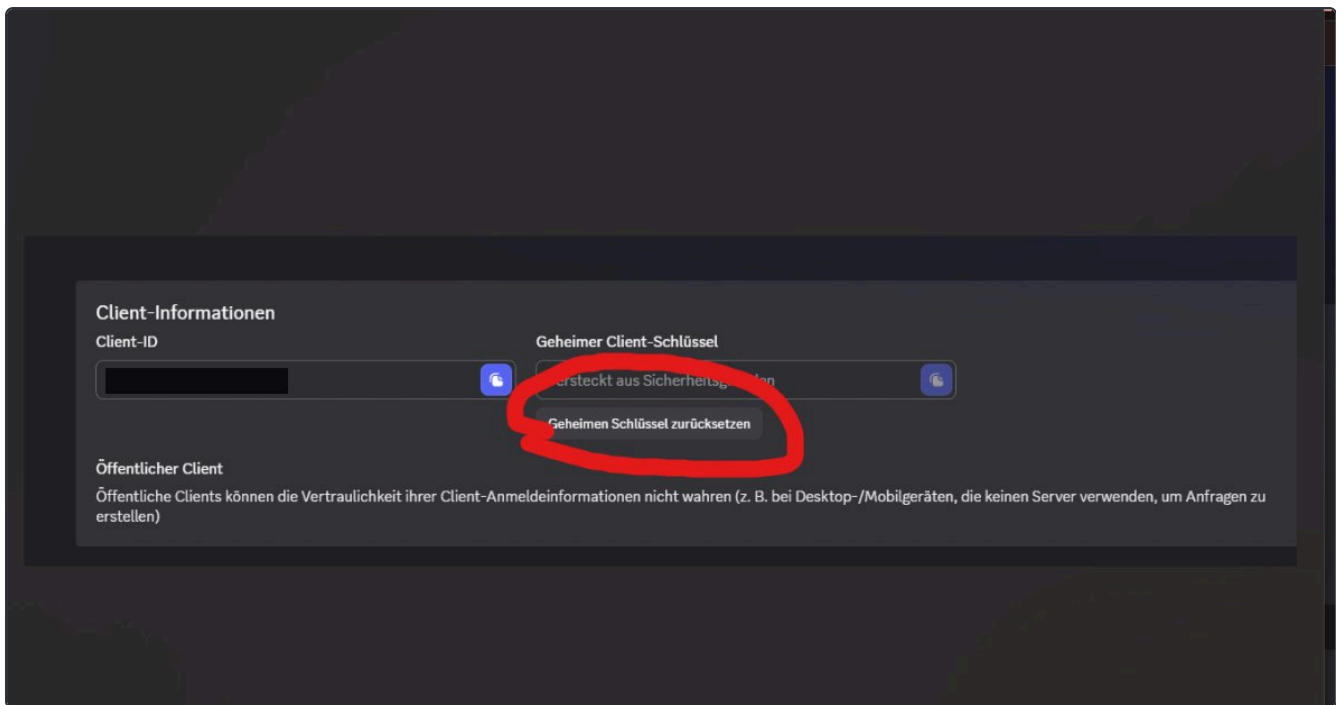
Generiere einen Einladungslink für deine Anwendung, indem du die Anwendungsbereiche und Berechtigungen auswählst, die sie benötigt, um zu funktionieren. Teile dann die URL mit anderen!

Anwendungsbereiche

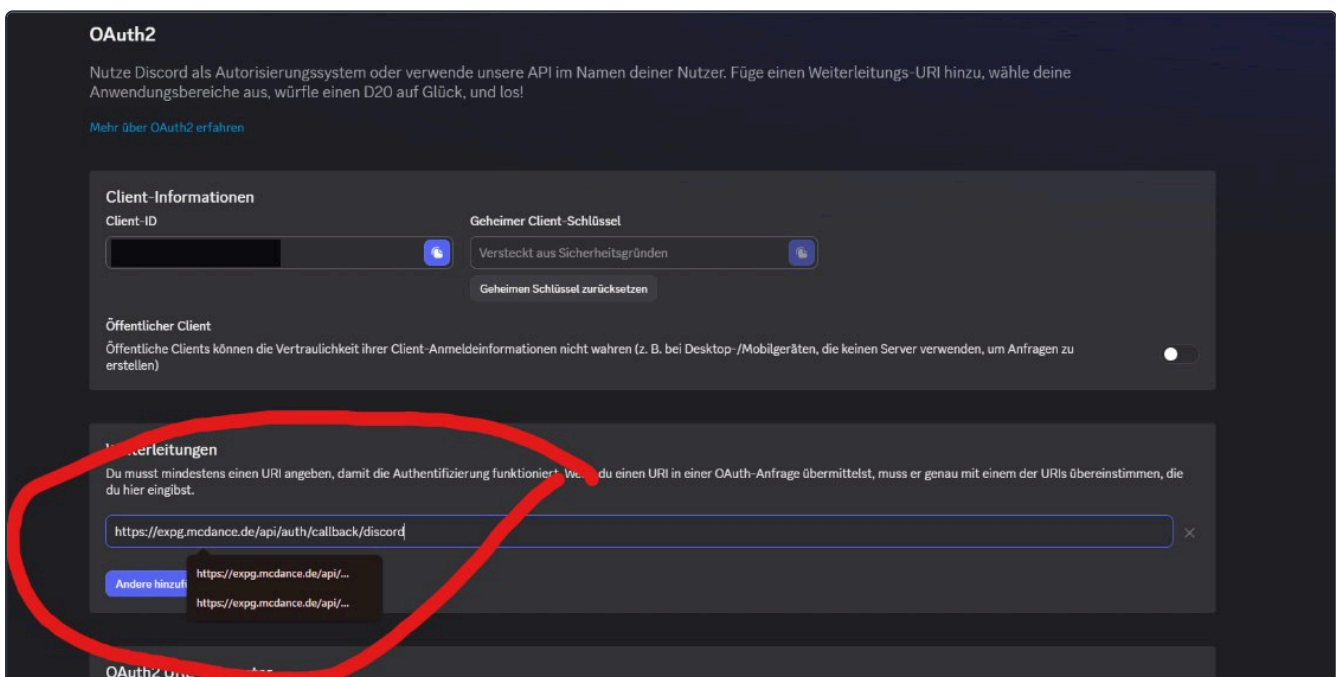
☐ identify	☐ email	☐ connections
☐ guilds	☐ guilds.join	☐ guilds.members.read
☐ gdm.join	☐ bot	☐ rpc
☐ rpc.notifications.read	☐ rpc.voice.read	☐ rpc.voice.write
☐ rpc.video.read	☐ rpc.video.write	☐ rpc.screenshare.read
☐ rpc.screenshare.write	☐ rpc.activities.write	☐ webhook.incoming
☐ messages.read	☐ applications.builds.read	☐ applications.commands
☐ applications.store.update	☐ applications.entitlements	☐ role_connections.write
☐ openid		
☐ applications.commands.permissions.update		

Die Seite OAuth2: oben „Client-Informationen“, darunter „Weiterleitungen“. Der URL-Generator weiter unten wird nicht gebraucht.

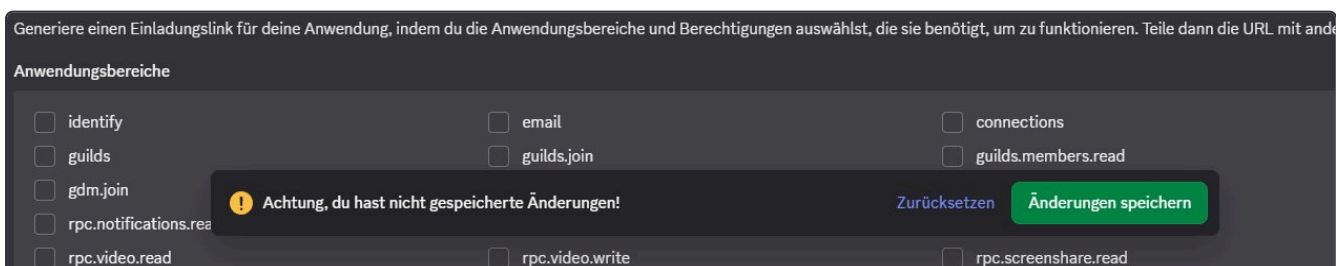
1. Klicke auf „Geheimen Schlüssel zurücksetzen“, bestätige und kopiere den Schlüssel **sofort** in deine Textdatei. Er wird nur einmal angezeigt. Das ist `AUTH_DISCORD_SECRET`.
2. Klicke unter „Weiterleitungen“ auf „Redirect hinzufügen“ und trage ein: `https://expg.mcdance.de/api/auth/callback/discord`. Die Adresse muss exakt so lauten, ohne Leerzeichen und ohne Schrägstrich am Ende. Statt `expg.mcdance.de` steht deine Adresse.
3. Klicke unten auf „Änderungen speichern“.



Auf „Geheimen Schlüssel zurücksetzen“ klicken, den Schlüssel kopieren und in einer Textdatei zwischenspeichern.



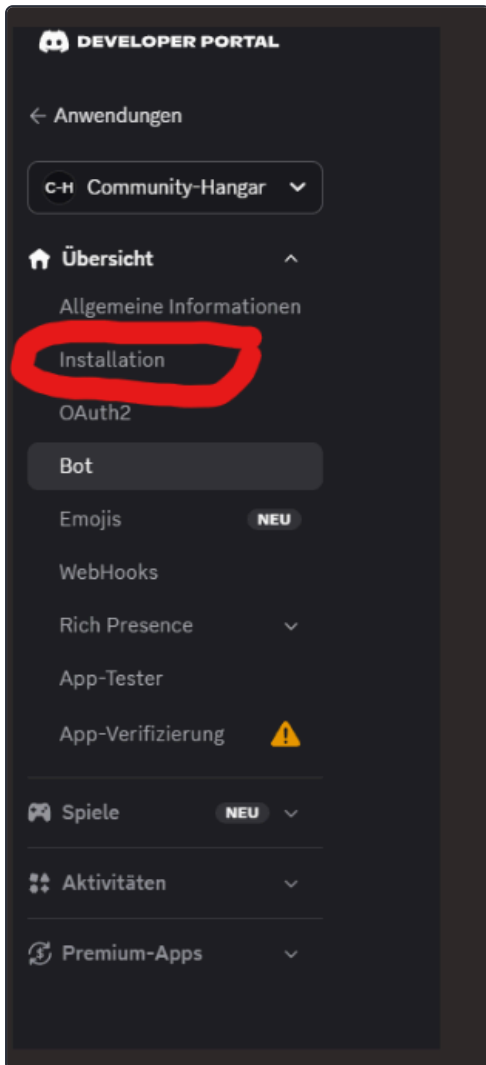
Die Redirect-Adresse eintragen. Die Vorschläge des Browsers unter dem Feld nicht anklicken (Esc drücken).



Nicht vergessen: Änderungen speichern.

2.4 Installation: Install Link auf „Keine“

Bei einem **privaten Bot** verlangt Discord, dass es keinen Standard-Einladungslink gibt. Stelle das deshalb **vor** den Bot-Einstellungen ein. Sonst meldet das Portal beim Speichern des privaten Bots: „*Private Anwendungen können keinen Standard-Autorisierungslink haben*“.



Menü links: Installation.

- 1 Wähle im Dropdown „Installations-Link“ den Eintrag „Keine“ (statt „Von Discord bereitgestellter Link“).
- 2 Nimm den Haken bei **Nutzerinstallation** heraus. Es bleibt nur die **Gildeninstallation**: Der Bot soll nur auf Servern laufen.
- 3 Speichere unten mit „Änderungen speichern“.

Installation

Wähle aus, wie Nutzer deine App installieren. Erstelle einen Installations-Link, wähle den Installationskontext, den du unterstützen möchtest und lege die Anwendungsbereiche und Berechtigungen fest, die du anfordern möchtest.

Installationskontexte
Wähle die Installationskontexte aus, die deine App unterstützt. Apps können sowohl für Nutzer als auch für Gilden installiert werden.

Methoden auswählen
Wähle die Installationskontexte aus, die deine App unterstützt. Apps können sowohl für Nutzer als auch für Gilden installiert werden.

☒ Nutzerinstallation
☒ Gildeninstallation

Installations-Link
Verwende unseren bereitgestellten Installations-Link oder registriere einen benutzerdefinierten. Wenn du einen benutzerdefinierten Link wählst, werden Benutzer, die deine App hinzufügen, anstelle des „App hinzufügen“-Ablaufs in Discord zu deiner URL weitergeleitet.

Von Discord bereitgestellter Link

https://discord.com/oauth2/authorize?client_id=155700512114869790

Standard-Installationseinstellungen
Wähle die Standardanwendungsbereiche und Berechtigungen, für die deine App standardmäßig Zugriff anfordern wird.

Nutzerinstallation
Nutzer können deine App zu ihrem Account hinzufügen und überall auf Discord verwenden.

Anwendungsbereiche
applications.commands

Gildeninstallation
Nutzer können deine App zu einer Gilde hinzufügen, wodurch sie Berechtigungen erhält, um in dieser Gilde Aktionen auszuführen.

Anwendungsbereiche
applications.commands

Installation: Das Dropdown „Installations-Link“ auf „Keine“ stellen, nur Gildeninstallation angehakt lassen (hier vor der Änderung).

 **Achtung, du hast nicht gespeicherte Änderungen!**

[Zurücksetzen](#) [Änderungen speichern](#)

Speichern nicht vergessen.

2.5 Bot: Token, Intent und privat schalten

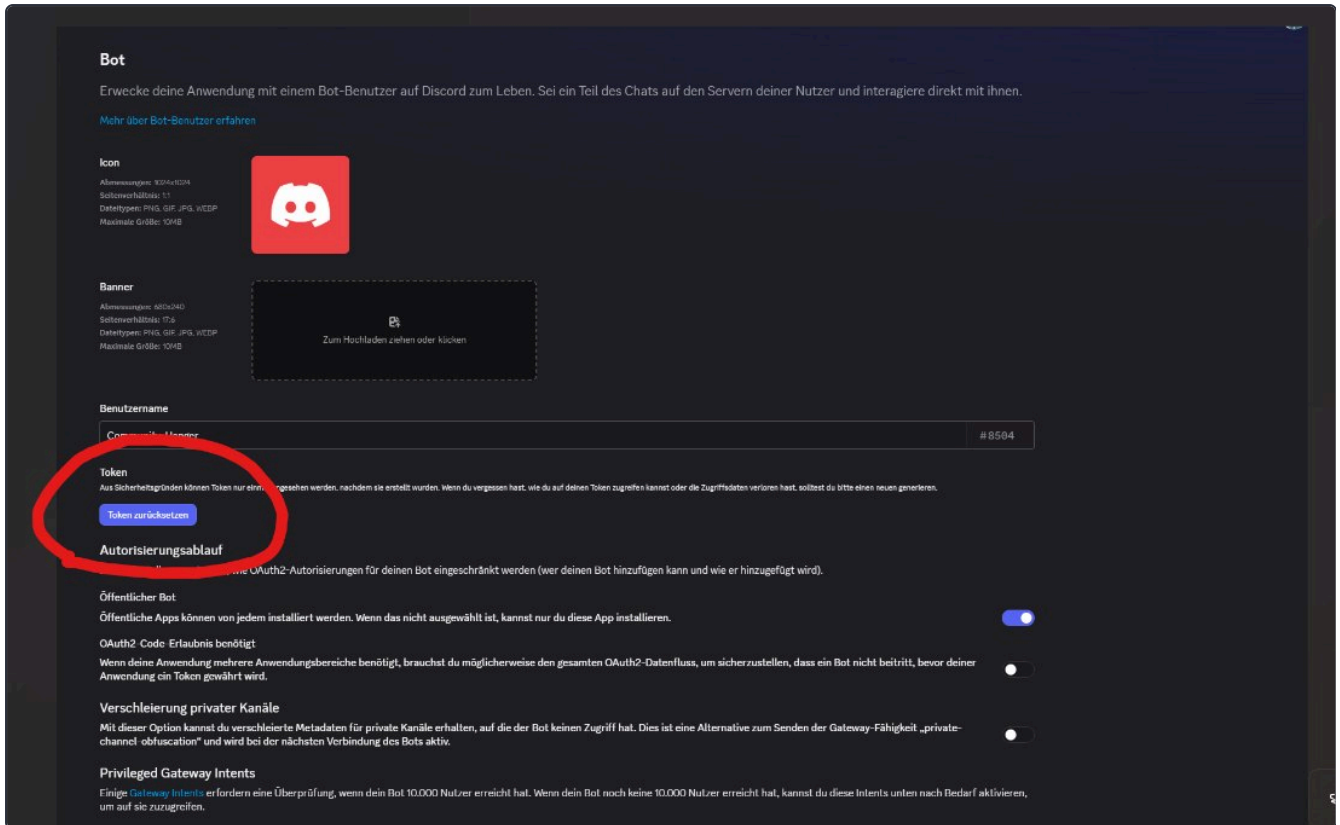
OAuth2

Bot

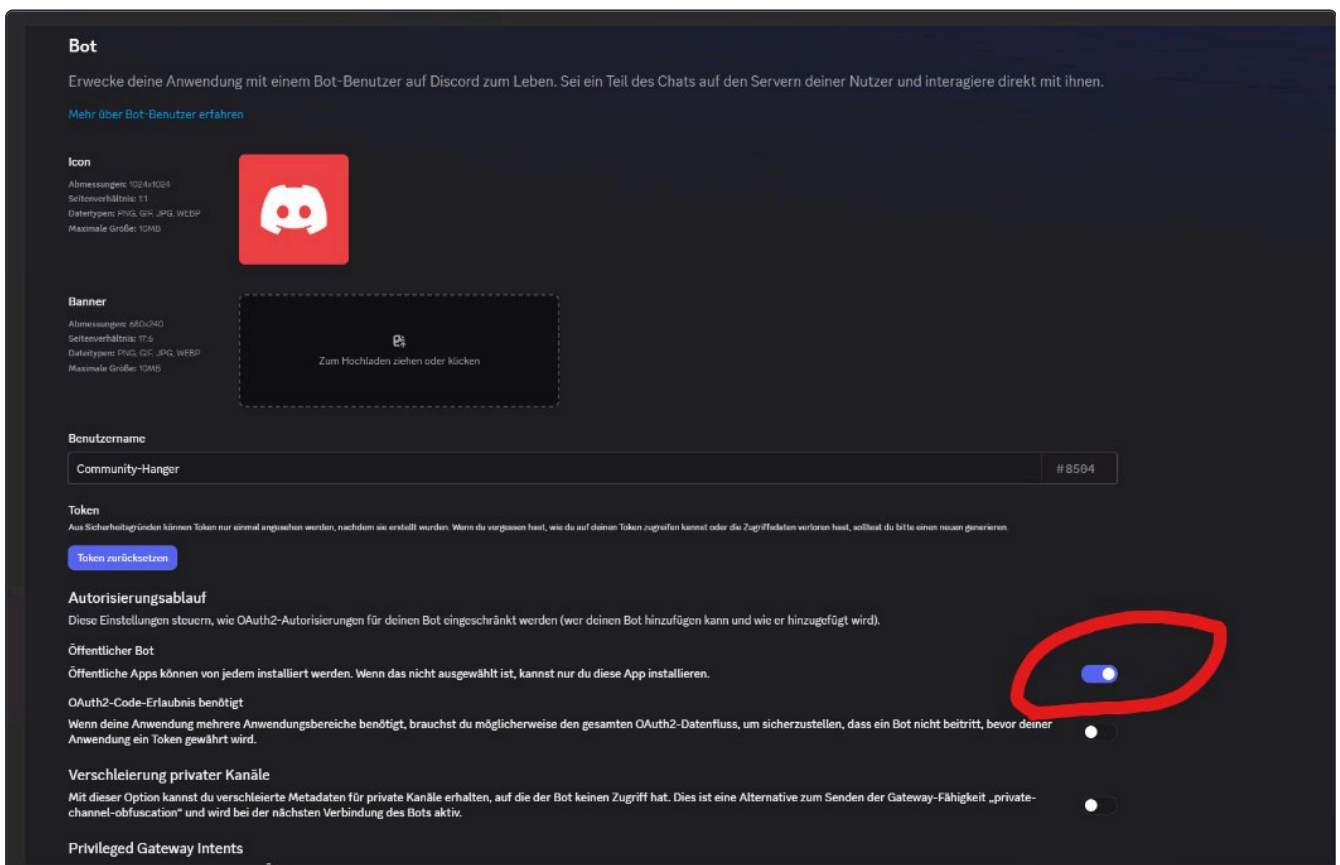
Emojis **NEU**

Menü links: Bot.

- 1 Klicke auf „Token zurücksetzen“, bestätige und kopiere den Token **sofort** in deine Textdatei. Er wird nur einmal angezeigt. Das ist `DISCORD_BOT_TOKEN`.
- 2 Scrolle zu **Öffentlicher Bot** und schalte ihn **aus** (grau). Dann kann nur du den Bot auf Server einladen.
- 3 Scrolle zu **Privileged Gateway Intents** und schalte den **Server Members Intent** ein (blau). Ohne ihn kann der Bot die Mitgliederliste nicht lesen. **Message Content Intent** und **Presence Intent** bleiben aus.
- 4 Speichere unten mit „Änderungen speichern“.



Bot: „Token zurücksetzen“ klicken und den Token sofort in die Textdatei kopieren.



„Öffentlicher Bot“ ausschalten (auf dem Bild ist der Schalter noch an, der rote Kreis markiert ihn).

Server Members Intent

Erforderlich, damit dein Bot Events empfängt, die unter `GUILD_MEMBERS` aufgeführt werden.

HINWEIS: Sobald dein Bot 10.000 oder mehr Nutzer erreicht, wird dafür eine Überprüfung nötig. [Hier erfährst du mehr](#)



Message Content Intent

Server Members Intent einschalten (hier noch aus). Der Hinweis zur Überprüfung ab 10.000 Nutzern betrifft euch nicht.

tents

rde



Achtung, du hast nicht gespeicherte Änderungen!

[Zurücksetzen](#)

[Änderungen speichern](#)

nten nach Be

Änderungen speichern.

Der **Benutzername** des Bots steht auf der Seite Bot und ist vom Namen der Anwendung getrennt. Er erscheint in Discord; prüfe die Schreibweise, bevor du speicherst.

2.6 Deine Discord-ID und die Server-ID

Zwei weitere Zahlen brauchst du aus Discord selbst. Schalte dazu in Discord unter **Einstellungen** → **Erweitert** den **Entwicklermodus** ein.

- **Deine Discord-ID** (`SERVER_ADMIN_DISCORD_ID` , damit erhältst du Zugang zum Bereich „Server-Admin“): Rechtsklick auf dein Profil → *Nutzer-ID kopieren*.
- **Die Server-ID** deines Discord-Servers (`ONBOARDING_GUILD_IDS` , nur dieser Server darf eine Orga anlegen): Rechtsklick auf den Server → *Server-ID kopieren*.

3 Dateien auf den Server legen

Melde dich auf deinem Server an (wie das geht, weißt du selbst am besten) und gib die folgenden Befehle der Reihe nach ein.

Befehl 1: Ordner anlegen und hineinwechseln

```
mkdir -p ~/community-hangar && cd ~/community-hangar
```

- `mkdir -p ~/community-hangar` legt den Ordner `community-hangar` in deinem Benutzerverzeichnis an. `-p` verhindert einen Fehler, falls es ihn schon gibt.
- `&&` führt den nächsten Befehl nur aus, wenn der erste geklappt hat.
- `cd ~/community-hangar` wechselt in diesen Ordner. Alle weiteren Befehle laufen dort.

Befehl 2: Docker-Compose-Datei herunterladen

```
curl -O https://raw.githubusercontent.com/mackedanz/community-hangar-1/main/docker-compose.yml
```

`curl` lädt eine Datei aus dem Internet, `-O` speichert sie unter ihrem Namen. Die Datei beschreibt, welche Container Docker startet (die App und die Datenbank) und welche Daten sie behalten. Fehlt `curl` („command not found“), installiere es mit `sudo apt install curl`.

Befehl 3: Vorlage für die Einstellungen herunterladen

```
curl -o .env https://raw.githubusercontent.com/mackedanz/community-hangar-1/main/.env.docker.example
```

`-o .env` (kleines o) speichert die Vorlage unter dem Namen `.env`. Diese Datei enthält später deine Einstellungen und Geheimnisse. Sie beginnt mit einem Punkt und ist deshalb unsichtbar; `ls -la` zeigt sie.

```
root@Docker:~# mkdir -p ~/community-hangar && cd ~/community-hangar
root@Docker:~/community-hangar# curl -O https://raw.githubusercontent.com/mackedanz/community-hangar-1/main/docker-compose.yml
curl -o .env https://raw.githubusercontent.com/mackedanz/community-hangar-1/main/.env.docker.example
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 3215 100 3215  0  0  10355    0 --:--:-- --:--:-- --:--:-- 10370
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 2276 100 2276  0  0  9395    0 --:--:-- --:--:-- --:--:-- 9404
root@Docker:~/community-hangar# ls -la
total 16
drwxr-xr-x  2 root root 4096 Oct  6 12:43 .
drwx----- 10 root root 4096 Oct  6 12:43 ..
-rw-r--r--  1 root root 2276 Oct  6 12:43 .env
-rw-r--r--  1 root root 3215 Oct  6 12:43 docker-compose.yml
root@Docker:~/community-hangar#
```

So sollte es aussehen: Die beiden Fortschrittsstabellen von curl sind normal. Am Ende stehen `.env` und `docker-compose.yml` im Ordner.

4 Einstellungen eintragen (.env)

```
nano .env
```

```
root@Docker:~/community-hangar# nano .env
```

Die Datei mit dem Texteditor nano öffnen.

```
GNU nano 7.2 .env
# Einstellungen für den Docker-Stack. In ".env" umbenennen und ausfüllen.

# ^öffentliche Adresse ohne https://, z. B. hangar.example.de
DOMAIN=

# Discord Developer Portal -> eure Anwendung -> OAuth2
# Dort unter "Redirects" eintragen: https://<DOMAIN>/api/auth/callback/discord
AUTH_DISCORD_ID=
AUTH_DISCORD_SECRET=

# Passwort der Datenbank (frei wählbar, nur für die Verbindung zwischen App und Datenbank)
DB_PASSWORD=

# Discord-ID des Betreibers: nur dieser Nutzer sieht den Bereich /admin (Orgas löschen, Server sperren).
# Mehrere IDs mit Komma trennen (z. B. für eine "Bergabe"). Leer = kein Admin-Bereich.
# Wechsel: ID hier ändern und "docker compose up -d" ausführen, ein Neubau ist nicht nötig.
SERVER_ADMIN_DISCORD_ID=

# Optional: Links zu Impressum und Datenschutzerklärung des Betreibers (öffnen in neuem Tab).
# Ohne LEGAL_PRIVACY_URL zeigt der Footer die eingebaute Datenschutzseite (Platzhalter);
# ohne LEGAL_IMPRINT_URL erscheint kein Impressum-Link.
LEGAL_IMPRINT_URL=
LEGAL_PRIVACY_URL=

# Optional: Katalog-Aktualisierung in Sekunden (Standard: 604800 = einmal pro Woche)
# CATALOG_SYNC_INTERVAL_SECONDS=604800

# Optional: Onboarding-Bot (Discord Developer Portal -> eure Anwendung)
# General Information -> Public Key => DISCORD_PUBLIC_KEY
# Bot -> Token => DISCORD_BOT_TOKEN
# Bot -> "Server Members Intent" einschalten, "Public Bot" ausschalten
# General Information -> "Interactions Endpoint URL": https://<DOMAIN>/discord/interactions
# Einmalig die Befehle registrieren: docker compose exec app php bin/register-commands.php
DISCORD_PUBLIC_KEY=
DISCORD_BOT_TOKEN=

# Optional: 1 = nur Mitglieder auf der Zugangsliste des Bots (und Server-Admins) dürfen sich anmelden.
# Erst einschalten, wenn der Bot die Orga eingerichtet hat, sonst sperrt ihr euch aus.
# LOGIN_REQUIRES_ALLOWLIST=0

# Port auf dem Server, an den der Reverse-Proxy (z. B. Nginx Proxy Manager) weiterleitet (Standard: 8181)
# HANGAR_PORT=8181

# Bei öffentlichem Bot dringend setzen: nur diese Discord-Server (IDs, mit Komma getrennt) dürfen per /einrichten
# eine Orga anlegen. Leer = jeder Server, auf dem der Bot ist. Server-ID: Discord-Entwicklermodus, Rechtsklick auf den Server.
# ONBOARDING_GUILD_IDS=
```

Die Vorlage. Die Umlaute sehen im Terminal kaputt aus („f r“): Das ist nur die Darstellung und kein Fehler.

Vier Zeilen sind in der Vorlage **auskommentiert** (# am Anfang):

`DISCORD_PUBLIC_KEY`, `DISCORD_BOT_TOKEN`, `LOGIN_REQUIRES_ALLOWLIST` und `ONBOARDING_GUILD_IDS`. Entferne dort das #, sonst werden sie ignoriert.

```

GNU nano 7.2
# Einstellungen für den Docker-Stack. In ".env" umbenennen und ausfüllen.

# Öffentliche Adresse ohne https://, z. B. hangar.example.de
DOMAIN=

# Discord Developer Portal -> eure Anwendung -> OAuth2
# Dort unter "Redirects" eintragen: https://<DOMAIN>/api/auth/callback/discord
AUTH_DISCORD_ID=
AUTH_DISCORD_SECRET=

# Passwort der Datenbank (frei wählbar, nur für die Verbindung zwischen App und Datenbank)
DB_PASSWORD=

# Discord-ID des Betreibers: nur dieser Nutzer sieht den Bereich /admin (Orgas löschen, Server sperren).
# Mehrere IDs mit Komma trennen (z. B. für eine Übergabe). Leer = kein Admin-Bereich.
# Wechsel: ID hier ändern und "docker compose up -d" ausführen, ein Neubau ist nicht nötig.
SERVER_ADMIN_DISCORD_ID=

# Optional: Links zu Impressum und Datenschutzerklärung des Betreibers (öffnen in neuem Tab).
# Ohne LEGAL_PRIVACY_URL zeigt der Footer die eingebaute Datenschutzseite (Platzhalter);
# ohne LEGAL_IMPRINT_URL erscheint kein Impressum-Link.
LEGAL_IMPRINT_URL=
LEGAL_PRIVACY_URL=

# Optional: Katalog-Aktualisierung in Sekunden (Standard: 604800 = einmal pro Woche)
# CATALOG_SYNC_INTERVAL_SECONDS=604800

# Optional: Onboarding-Bot (Discord Developer Portal -> eure Anwendung)
# General Information -> Public Key => DISCORD_PUBLIC_KEY
# Bot -> Token => DISCORD_BOT_TOKEN
# Bot -> "Server Members Intent" einschalten, "Public Bot" ausschalten
# General Information -> "Interactions Endpoint URL": https://<DOMAIN>/discord/interactions
# Einmalig die Befehle registrieren: docker compose exec app php bin/register-commands.php
DISCORD_PUBLIC_KEY=
DISCORD_BOT_TOKEN=

# Optional: 1 = nur Mitglieder auf der Zugangsliste des Bots (und Server-Admins) dürfen sich anmelden.
# Erst einschalten, wenn der Bot die Orga eingerichtet hat, sonst sperrt ihr euch aus.
LOGIN_REQUIRES_ALLOWLIST=0

# Port auf dem Server, an den der Reverse-Proxy (z. B. Nginx Proxy Manager) weiterleitet (Standard: 8181)
# HANGAR_PORT=8181

# Bei öffentlichem Bot dringend setzen: nur diese Discord-Server (IDs, mit Komma getrennt) dürfen per /einrichten
# eine Orga anlegen. Leer = jeder Server, auf dem der Bot ist. Server-ID: Discord-Entwicklermodus, Rechtsklick auf den Server.
ONBOARDING_GUILD_IDS=

```

So sieht die Vorlage aus, wenn die Zeilen aktiviert sind. Jetzt trägst du deine Werte ein.

Zeile	Wert
DOMAIN=	deine Adresse ohne https://, z. B. expg.mcdance.de
AUTH_DISCORD_ID=	Anwendungs-ID
AUTH_DISCORD_SECRET=	Geheimer Client-Schlüssel
DB_PASSWORD=	frei wählbar, lang und zufällig. Erzeugen mit: openssl rand -base64 24 tr -d '/+='
SERVER_ADMIN_DISCORD_ID=	deine Discord-ID
DISCORD_PUBLIC_KEY=	Öffentlicher Schlüssel
DISCORD_BOT_TOKEN=	Bot-Token
LOGIN_REQUIRES_ALLOWLIST=	1: Nur wer auf der Zugangsliste des Bots steht (oder Server-Admin ist), darf sich anmelden
ONBOARDING_GUILD_IDS=	Server-ID deines Discord-Servers

Tipps zu nano: Mit den **Pfeiltasten** navigieren, die Maus funktioniert nicht. Zum Einfügen aus der Zwischenablage je nach Terminal Rechtsklick oder

Strg + **Umschalt** + **V**. Werte stehen direkt hinter dem **=**, ohne Leerzeichen und ohne Anführungszeichen. Speichern mit **Strg** + **0** und Enter, beenden mit **Strg** + **X**.

```

$ git nano 7.2
# Einstellungen f. r den Docker-Stack. In "env" umbenennen und auf "llm".
# "öffentliche Adresse ohne https://, z. B. hangar.example.de
DOMAIN=exp.mcdance.de

# Discord Developer Portal -> eure Anwendung -> OAuth2
# Durs autor "redirects" eintragen: https://<DOMAIN>/api/auth/callback/discord
AUTH_DISCORD_ID=
AUTH_DISCORD_SECRET=

# Passwort der Datenbank (frei w. hbar, nur f. r die Verbindung zwischen App und Datenbank)
DB_PASSWORD=

# Discord-ID des Betreibers: nur dieser Nutzer sieht den Bereich /admin (Orgas l. schen, Server sperren).
# Mehrere IDs mit Komma trennen (z. B. f. r eine "übergabe"). Leer = kein Admin-Bereich.
# Wechsel: ID hier ändern und "docker compose up -d" auf hren, ein Neubau ist nicht n. tlg.
SERVER_ADMIN_DISCORD_ID=

# Optional: Links zu Impressum und Datenschutzerklärung des Betreibers (öffnen in neuem Tab).
# Ohne LEGAL_PRIVACY_URL zeigt der Footer die eingebaute Datenschutzhilfe (Platzhalter);
# ohne LEGAL_IMPRINT_URL erscheint kein Impressum-Link.
LEGAL_IMPRINT_URL=
LEGAL_PRIVACY_URL=

# Optional: Katalog-Aktualisierung in Sekunden (Standard: 604800 = einmal pro Woche)
# CATALOG_SYNC_INTERVAL_SECONDS=604800

# Optional: Onboarding-Bot (Discord Developer Portal -> eure Anwendung)
# General Information -> Public Key => DISCORD_PUBLIC_KEY
# Bot -> Token => DISCORD_BOT_TOKEN
# Bot -> "Server Members Intent" einschalten, "Public Bot" ausschalten
# General Information -> "Interactions Endpoint URL": https://<DOMAIN>/discord/interactions
# Einmalig die Befehle registrieren: docker compose exec app php bin/register-commands.php
DISCORD_PUBLIC_KEY=
DISCORD_BOT_TOKEN=

# Optional: l = nur Mitglieder auf der Zugangsliste des Bots (und Server-Admins) d. rfen sich anmelden.
# l = einschalten, wenn der Bot die Orga eingerichtet hat, sonst sperrt ihr euch aus.
LOGIN_REQUIRES_ALLOWLIST=1

# Port auf dem Server, an den der Reverse-Proxy (z. B. Nginx Proxy Manager) weiterleitet (Standard: 8181)
# HANGAR_PORT=8181

# Bei öffentlichen Bot dringend setzen: nur diese Discord-Server (IDs, mit Komma getrennt) d. rfen per /einrichten
# eine Orga anlegen. Leer = jeder Server, auf dem der Bot ist. Server-ID: Discord-Entwicklermodus, Rechtsklick auf den Server.
ONBOARDING_GUILD_IDS=

```

Die ausgefüllte Datei (Geheimnisse und Kennungen sind hier geschwärzt).

Wichtig: Mit `LOGIN_REQUIRES_ALLOWLIST=1` löscht jeder Abgleich der Mitgliederliste das Konto samt Hangar von allen, die nicht mehr auf einer Zugangsliste stehen. Du selbst bist über `SERVER_ADMIN_DISCORD_ID` immer ausgenommen. Mit `0` ist die Sperre aus.

Dateirechte setzen und prüfen

Danach darf nur noch der Besitzer die Datei lesen. Der Befehl gibt bei Erfolg nichts aus.

```
chmod 600 .env
```

```
root@Docker:~/community-hangan# chmod 600 .env
```

Der Befehl chmod.

Mit diesem Befehl prüfst du, ob alle Zeilen gefüllt sind. Er zeigt von jedem Wert nur die ersten drei Zeichen:

```
sed -E 's/(.{3}).*/=\1.../' .env | grep -v '^#' | grep -v '^$'
```

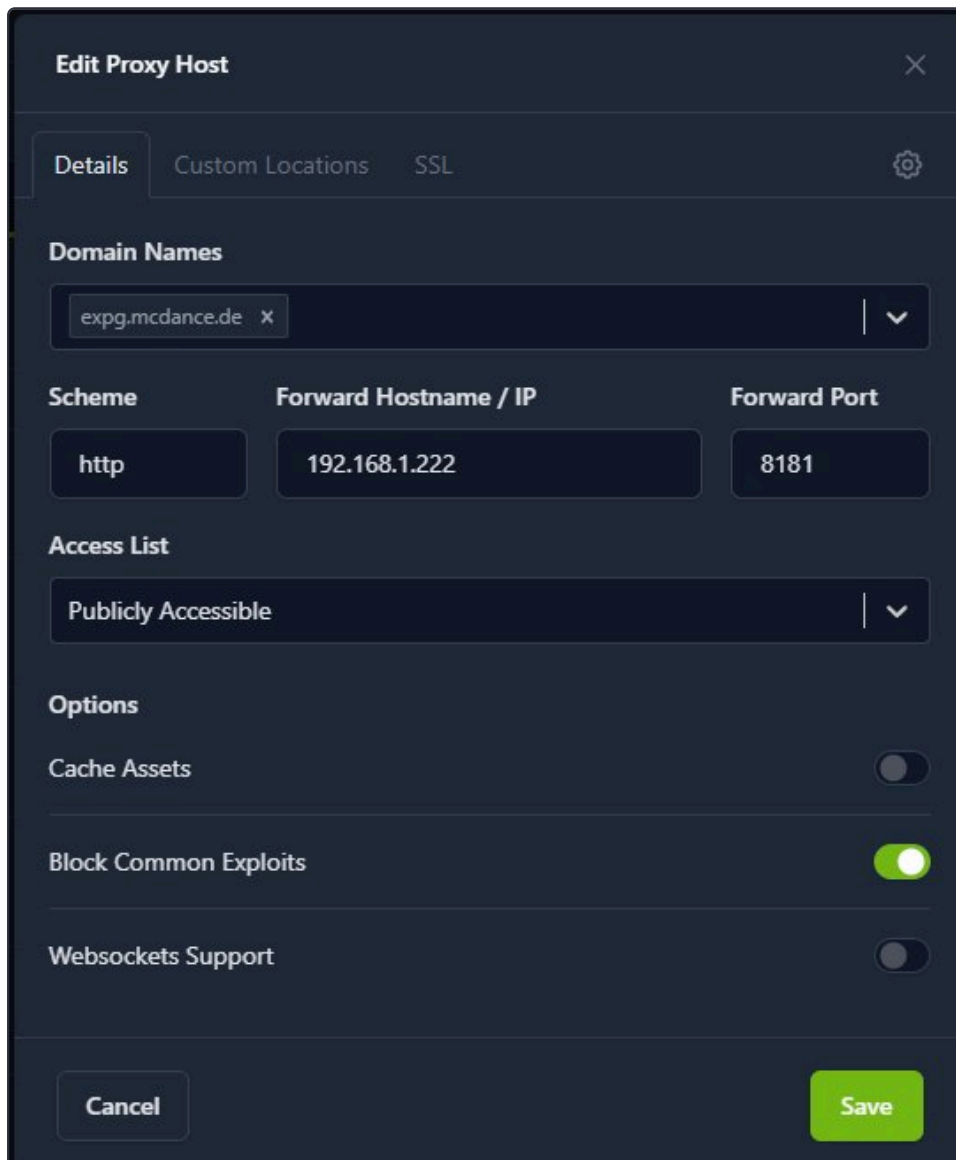
```
root@Docker:~/community-hangar# sed -E 's/(.{3}).*/=\1_/' .env | grep -v '^#' | grep -v '^$'
DOMAIN=exp...
AUTH_DISCORD_ID=155...
AUTH_DISCORD_SECRET=Nv9...
DB_PASSWORD=han...
SERVER_ADMIN_DISCORD_ID=399...
LEGAL_IMPRINT_URL=
LEGAL_PRIVACY_URL=
DISCORD_PUBLIC_KEY=7e7...
DISCORD_BOT_TOKEN=MTU...
LOGIN_REQUIRES_ALLOWLIST=1
ONBOARDING_GUILD_IDS=685...
root@Docker:~/community-hangar#
```

Alle Pflichtzeilen haben einen Wert. Nur LEGAL_IMPRINT_URL und LEGAL_PRIVACY_URL dürfen leer bleiben.

5 HTTPS einrichten (Reverse-Proxy)

Die App spricht nur HTTP. Davor muss ein Reverse-Proxy HTTPS und das Zertifikat übernehmen. Die `docker-compose.yml` gibt dafür den Port **8181** auf dem Server frei (änderbar mit `HANGAR_PORT`). Prüfe vorher, dass er frei ist: `ss -ltn | grep 8181` (keine Ausgabe = frei). Hier der Weg mit dem **Nginx Proxy Manager**. Ohne eigenen Proxy findest du einen Hinweis in Kapitel 13.

Reiter „Details“

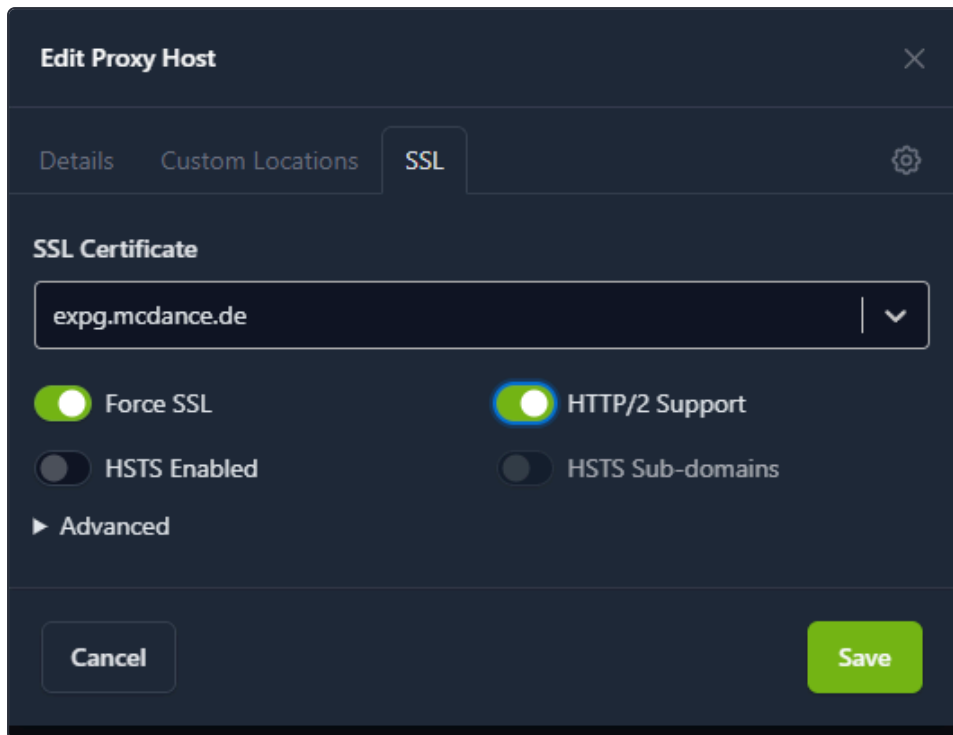


The screenshot shows the 'Edit Proxy Host' dialog box with the 'Details' tab selected. The dialog has a dark theme and a close button (X) in the top right corner. Below the tabs, there is a 'Domain Names' section with a text input field containing 'expg.mcdance.de' and a dropdown arrow. Below this is a table with three columns: 'Scheme', 'Forward Hostname / IP', and 'Forward Port'. The 'Scheme' column has a button labeled 'http'. The 'Forward Hostname / IP' column has a text input field containing '192.168.1.222'. The 'Forward Port' column has a text input field containing '8181'. Below the table is an 'Access List' section with a dropdown menu showing 'Publicly Accessible'. Below this is an 'Options' section with three toggle switches: 'Cache Assets' (disabled), 'Block Common Exploits' (enabled), and 'Websockets Support' (disabled). At the bottom of the dialog are two buttons: 'Cancel' and 'Save'.

Scheme	Forward Hostname / IP	Forward Port
http	192.168.1.222	8181

Proxy-Host: Domain, Schema http, die IP deines Servers, Port 8181. „Block Common Exploits“ an, „Cache Assets“ und „Websockets“ aus. Zugriffsliste: „Publicly Accessible“.

Reiter „SSL“



The screenshot shows the 'Edit Proxy Host' dialog box with the 'SSL' tab selected. The 'SSL Certificate' dropdown menu is set to 'expg.mcdance.de'. The 'Force SSL' and 'HTTP/2 Support' toggle switches are turned on, while 'HSTS Enabled' and 'HSTS Sub-domains' are turned off. An 'Advanced' section is collapsed. At the bottom, there are 'Cancel' and 'Save' buttons.

Edit Proxy Host [Close]

Details Custom Locations **SSL** [Settings]

SSL Certificate

expg.mcdance.de [Dropdown Arrow]

☒ Force SSL ☒ HTTP/2 Support

☐ HSTS Enabled ☐ HSTS Sub-domains

▶ Advanced

Cancel Save

SSL-Zertifikat wählen oder neu anfordern („Request a new certificate“ und die Bedingungen von Let’s Encrypt bestätigen). „Force SSL“ und „HTTP/2“ an, „HSTS“ aus. Dann „Save“.

Bis der Container läuft, zeigt die Domain **502 Bad Gateway**. Das ist normal.

6 Starten und prüfen

```
docker compose up -d
```

Docker legt Netzwerk, Volumes und Container an. „Waiting“ bei der Datenbank ist normal, die App startet, sobald sie bereit ist. Die Volumes `db-data` (Datenbank) und `ship-images` (Schiffsbilder) sind die dauerhaften Daten. Beim ersten Mal lädt Docker außerdem die Images herunter, das dauert deutlich länger.

```
root@Docker:~/community-hangar# docker compose up -d
[+] Running 4/5
✔ Network community-hangar_default Created 0.0s
✔ Volume community-hangar_db-data Created 0.0s
✔ Volume community-hangar_ship-images Created 0.0s
✔ Container community-hangar-db-1 Waiting 1.6s
✔ Container community-hangar-app-1 Created 0.0s
```

Docker legt Netzwerk, Volumes und Container an.

```
docker compose logs --tail 40 app
```

```
root@Docker:~/community-hangar# docker compose logs --tail 40 app
app-1 | Insgespielt: 001_init.sql, 002_onboarding.sql, 003_item_info_image_checked.sql, 004_ship_images_from_rsi.sql
app-1 | AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.25.0.3. Set the 'ServerName' directive globally to suppress this message
app-1 | AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.25.0.3; Set the 'ServerName' directive globally to suppress this message
app-1 | [Tue Oct 06 14:27:13.183317 2026] [mpm_prefork:notice] [pid 1:tid 1] AH00163: Apache/2.4.68 (Debian) configured -- resuming normal operations
app-1 | [Tue Oct 06 14:27:13.183346 2026] [core:notice] [pid 1:tid 1] AH00994: Command line: 'apache2 -D FOREGROUND'
app-1 | 127.0.0.1 - - [06/Oct/2026:14:27:17 +0000] "GET /healthz HTTP/1.1" 200 221 "-" "curl/8.14.1"
```

Beim ersten Start steht „Eingespielt“ mit den Datenbank-Migrationen. Nach einem Neustart ohne Änderungen steht „Datenbank ist aktuell.“. Die Apache-Warnung AH00558 ist harmlos. „GET /healthz ... 200“ ist der Gesundheitstest.

Nach etwa 20 Sekunden beginnt im Hintergrund der Abgleich des Schiffskatalogs aus der RSI Ship Matrix (ein bis zwei Minuten).

Auf die neueste Version bringen

```
docker compose pull && docker compose up -d
```

`pull` holt die neueste Version des Images, `up -d` startet die Container damit neu. `&&` führt den zweiten Befehl nur aus, wenn der erste geklappt hat. Das nutzt du auch später für jedes Update. Die Datenbank bleibt dabei erhalten.

```
root@Docker:~/community-hangar# docker compose pull && docker compose up -d
```

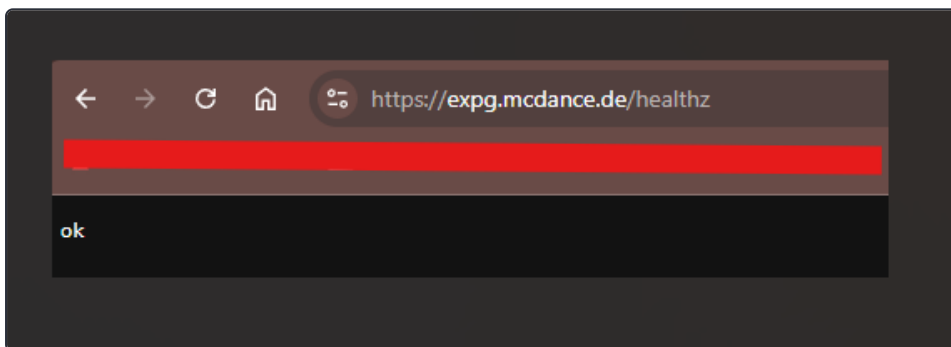
Der Befehl.

```
root@docker:~/community-hangar# docker compose pull && docker compose up -d
[*] Pulling 30/30
✓ app Pulled
  ✓ ec518c1e359 Already exists 2.3s
  ✓ 7a19cd40b05 Already exists 0.0s
  ✓ 4633f536cde Already exists 0.0s
  ✓ 8822cf316b2 Already exists 0.0s
  ✓ 53a3617988c Already exists 0.0s
  ✓ c9594de4676 Already exists 0.0s
  ✓ cde7d186a1a Already exists 0.0s
  ✓ 5a2aab6bd16 Already exists 0.0s
  ✓ 10ab328c1454 Already exists 0.0s
  ✓ 7a3652c884f1 Already exists 0.0s
  ✓ 4a256314083a Already exists 0.0s
  ✓ 8f209d1853df Already exists 0.0s
  ✓ 9d87683deba2 Already exists 0.0s
  ✓ 9286dad1b1e Already exists 0.0s
  ✓ 4f4fb70ba454 Already exists 0.0s
  ✓ 4c7811a287f0 Already exists 0.0s
  ✓ 87f7f2275a8a Already exists 0.0s
  ✓ 49c130c312ba Already exists 0.0s
  ✓ 139da4af1576 Already exists 0.0s
  ✓ 4986c6a4790 Pull complete 0.4s
  ✓ fafba8956f08 Pull complete 0.4s
  ✓ dc57389bc3e9 Pull complete 0.5s
  ✓ 50e10f51702f Pull complete 0.7s
  ✓ 2b311cd25f5b Pull complete 0.8s
  ✓ bdc5513b2c75 Pull complete 1.0s
  ✓ f7542b740894 Pull complete 1.1s
  ✓ ee055ffe7c5f Pull complete 1.1s
✓ db Pulled 0.9s
[*] Running 2/2
  ✓ Container community-hangar-db-1 Healthy 1.0s
  ✓ Container community-hangar-app-1 Started 1.9s
root@docker:~/community-hangar#
```

„Already exists“: schon vorhanden. „Pull complete“: neu geladen. „Healthy“ und „Started“: Die Container laufen.

Prüfen

Öffne im Browser <https://expg.mcdance.de/healthz>. Dort muss **ok** stehen. Erst wenn das klappt, geht es mit Discord weiter.



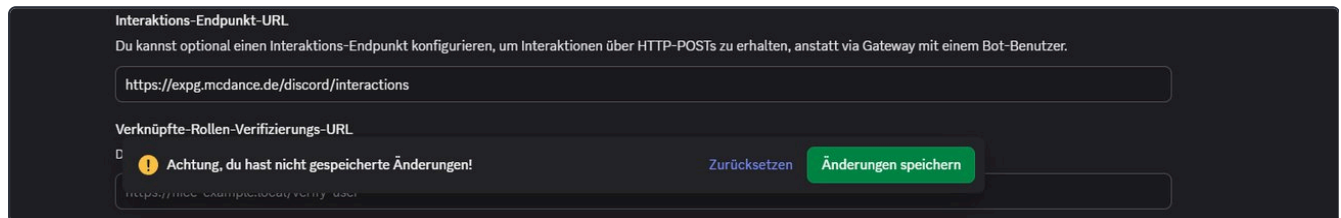
Zeigt die Seite „ok“, läuft die App, der Proxy ist richtig eingestellt und das Zertifikat greift.

7 Endpunkt bei Discord eintragen

Im Developer Portal: **Allgemeine Informationen** → **Interaktions-Endpunkt-URL**.

Trage ein (statt `expg.mcdance.de` deine Adresse) und speichere:

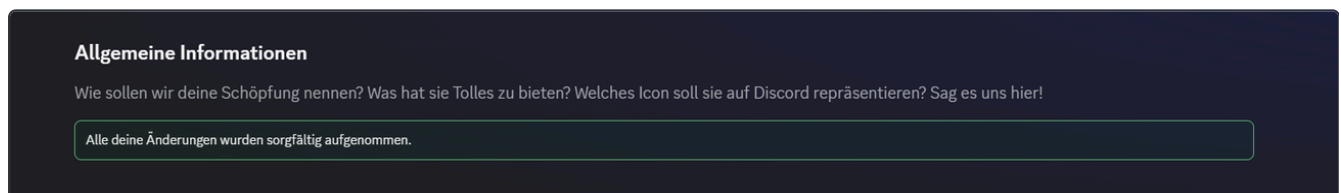
```
https://expg.mcdance.de/discord/interactions
```



The screenshot shows the 'Interaktions-Endpunkt-URL' configuration page in the Discord Developer Portal. It includes a text input field containing the URL 'https://expg.mcdance.de/discord/interactions'. Below this is a section for 'Verknüpfte-Rollen-Verifizierungs-URL' with a warning icon and the text 'Achtung, du hast nicht gespeicherte Änderungen!'. To the right of the warning are two buttons: 'Zurücksetzen' and 'Änderungen speichern'.

Interaktions-Endpunkt-URL eintragen und auf „Änderungen speichern“ klicken.

Beim Speichern schickt Discord eine Testanfrage an deinen Server. Klappt sie, erscheint die grüne Meldung:



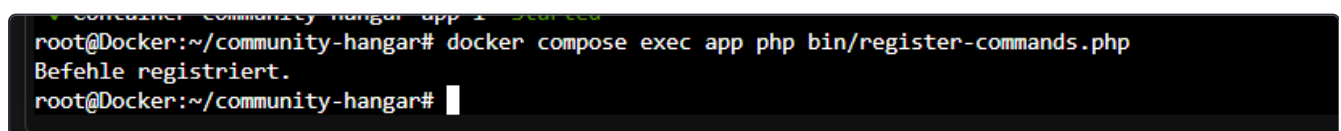
The screenshot shows the 'Allgemeine Informationen' section of the Discord Developer Portal. It contains a green message box that reads: 'Alle deine Änderungen wurden sorgfältig aufgenommen.'

Erfolg: Discord hat den Endpunkt geprüft und gespeichert.

Meldet das Portal einen Fehler, ist meist `DISCORD_PUBLIC_KEY` falsch oder die Seite nicht erreichbar (Kapitel 6 prüfen). Nach jeder Änderung an der `.env` gilt:
`docker compose up -d`.

8 Befehle registrieren

```
docker compose exec app php bin/register-commands.php
```



The screenshot shows a terminal window with the command `docker compose exec app php bin/register-commands.php` being executed. The output shows the command being registered successfully, with the text 'Befehle registriert.'

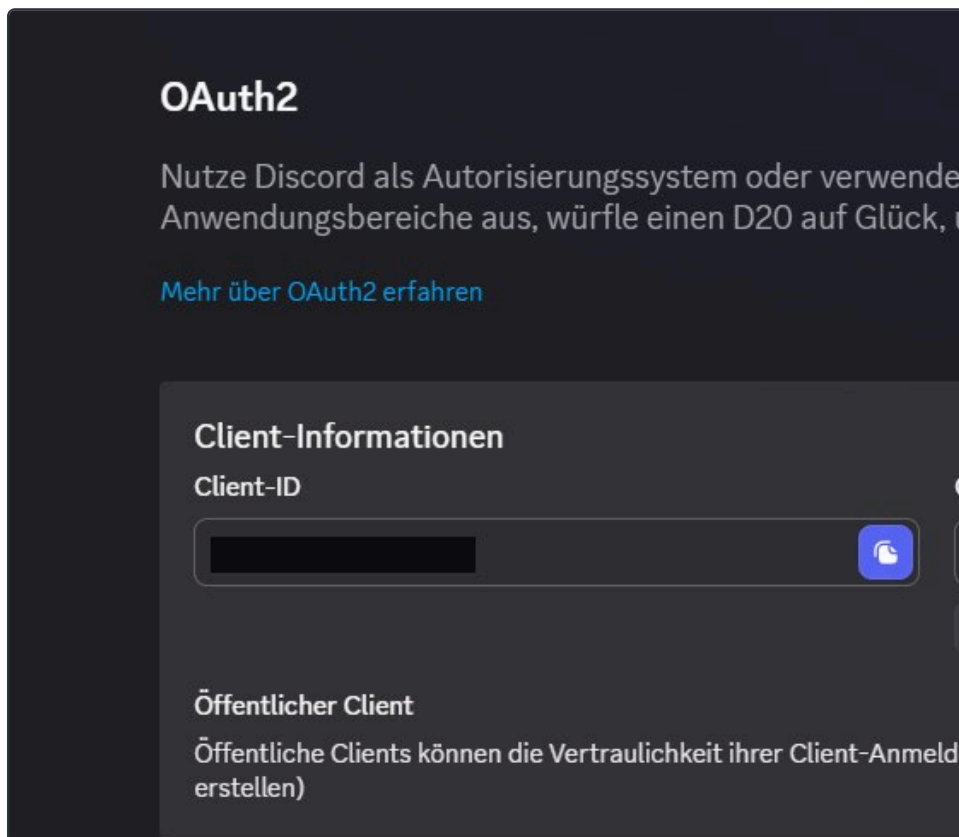
Erscheint „Befehle registriert.“, kennt Discord die Slash-Befehle `/einrichten` und `/abgleichen`.

9 Bot auf den Server einladen

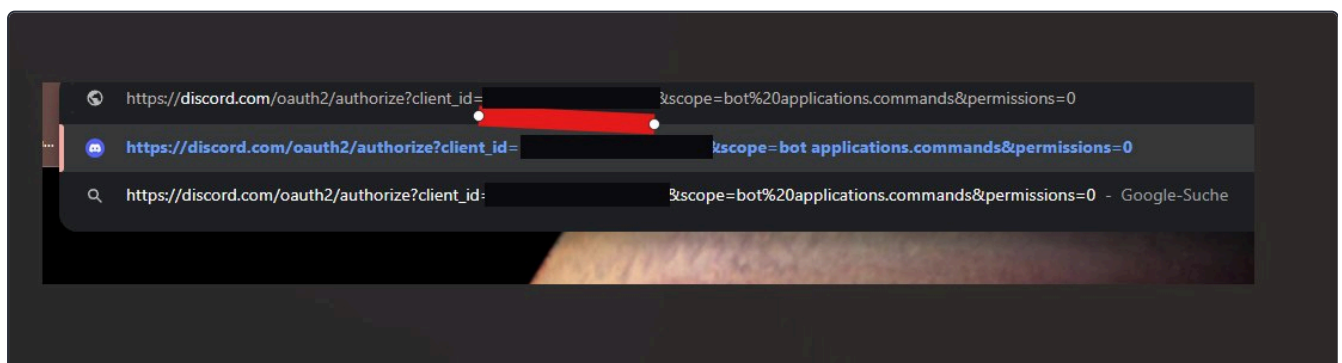
Öffne diesen Link im Browser und ersetze `<DEINE CLIENT-ID>` durch deine eigene ID:

```
https://discord.com/oauth2/authorize?client_id=<DEINE CLIENT-ID>&scope=bot%20applications.commands&permissions=0
```

Die Client-ID findest du im Developer Portal unter **OAuth2** im Feld **Client-ID**. Das ist dieselbe Zahl wie die Anwendungs-ID.



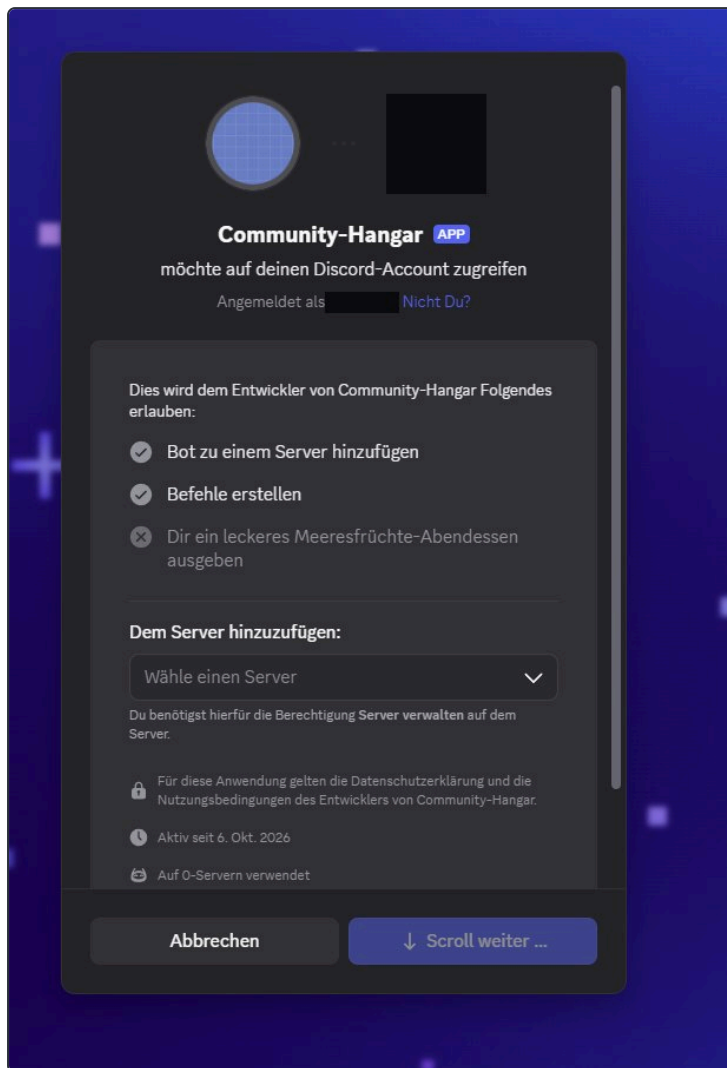
Die Client-ID auf der Seite OAuth2 (hier geschwärzt).



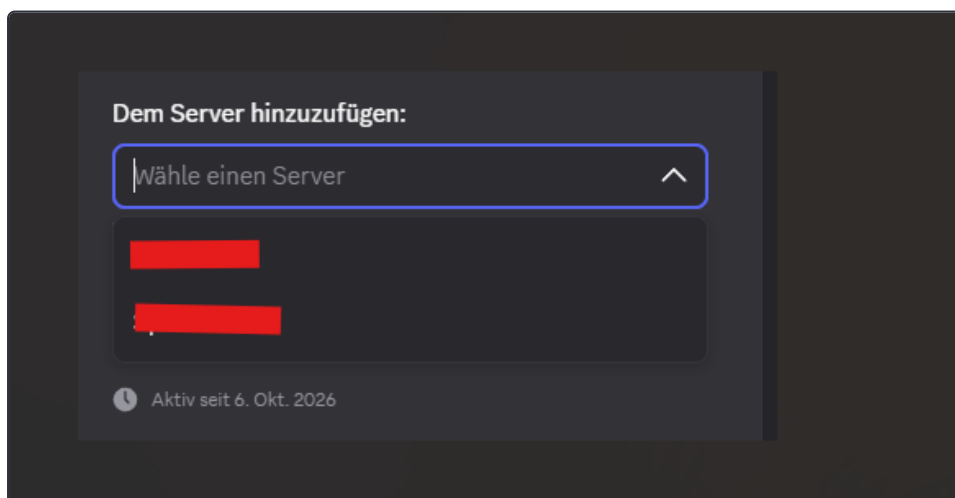
Den Link in die Adresszeile einfügen und mit Enter bestätigen, nicht auf einen Suchvorschlag klicken.

Der Bot braucht keine Rechte (`permissions=0`), nur zwei Zugriffe: Bot hinzufügen und Befehle erstellen. Wähle im Dropdown deinen Server. Es erscheinen nur Server,

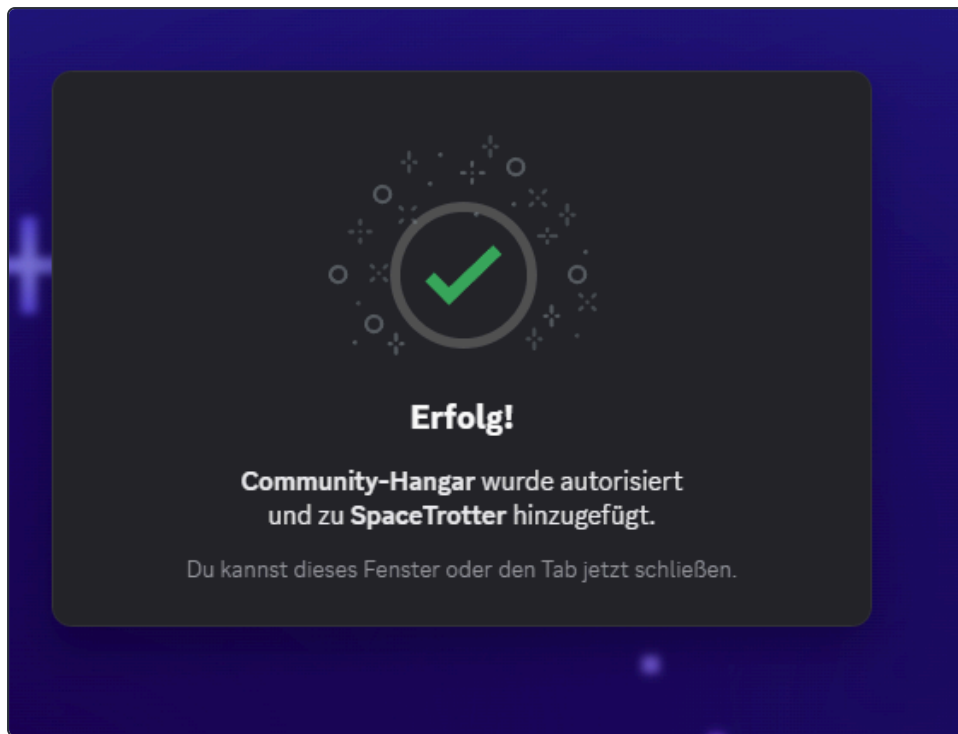
auf denen du das Recht „Server verwalten“ hast. Wähle den Server, dessen ID du in `ONBOARDING_GUILD_IDS` eingetragen hast. Eine eventuelle Captcha-Abfrage bestätigst du selbst.



Einladungsseite: scrollen, bis „Autorisieren“ aktiv wird.

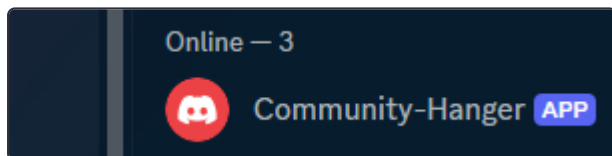


Dem Server hinzufügen: den Server wählen (hier geschwärzt).



Erfolg: Der Bot ist auf deinem Server.

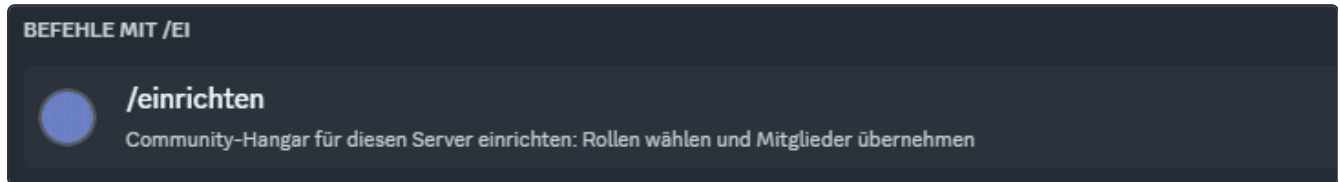
Der Bot erscheint in der Mitgliederliste des Servers (online oder offline, beides ist in Ordnung). Er antwortet nur auf Befehle.



Der Bot in der Mitgliederliste.

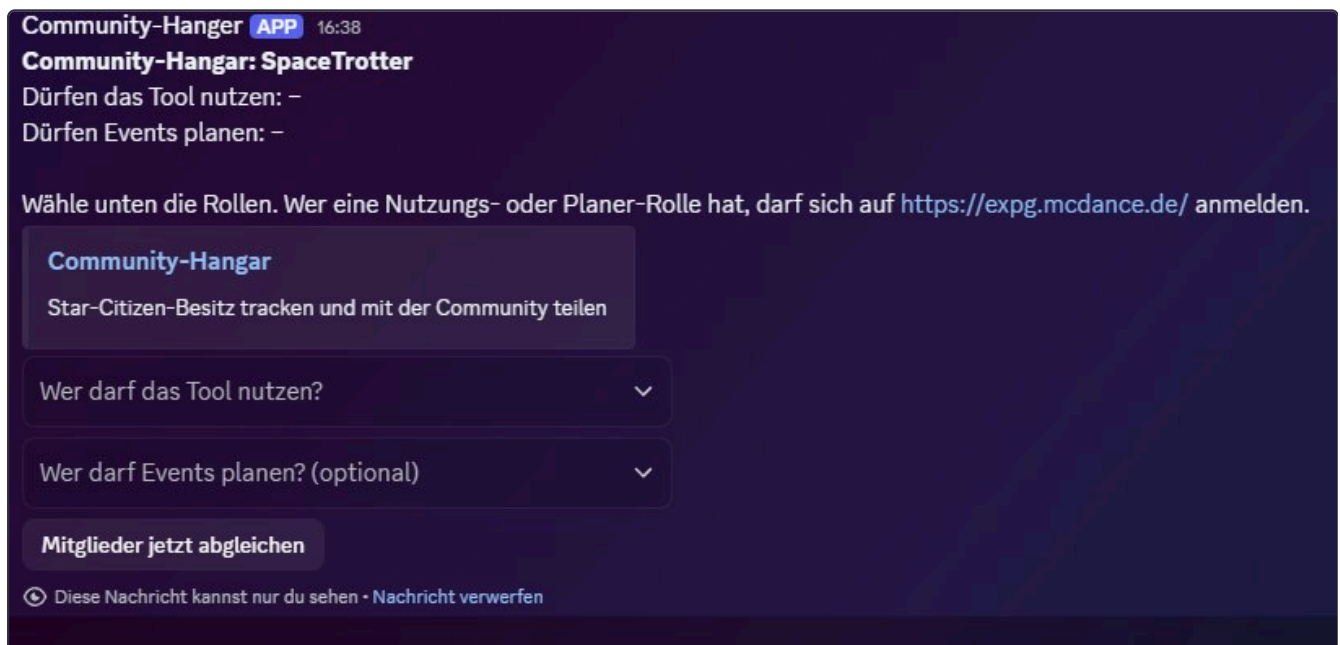
10 Orga einrichten

Tippe in Discord in einem beliebigen Kanal `/einrichten` (als Admin oder mit „Server verwalten“). Erscheint der Befehl nicht, lade Discord mit `Strg` + `R` neu; es kann ein paar Minuten dauern.



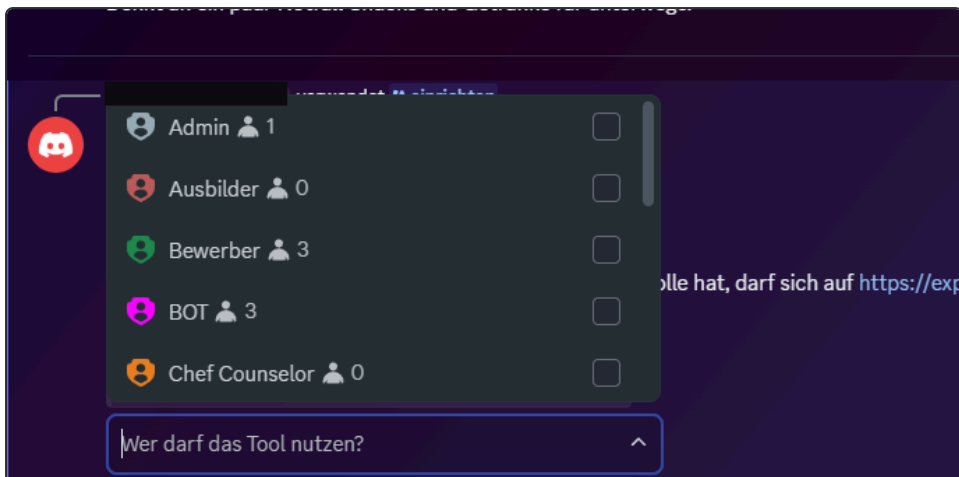
Der Befehl `/einrichten` im Vorschlagsmenü.

Es erscheint eine Nachricht, die nur du siehst. Die Orga wird mit dem Servernamen angelegt, du stehst automatisch auf der Zugangsliste.

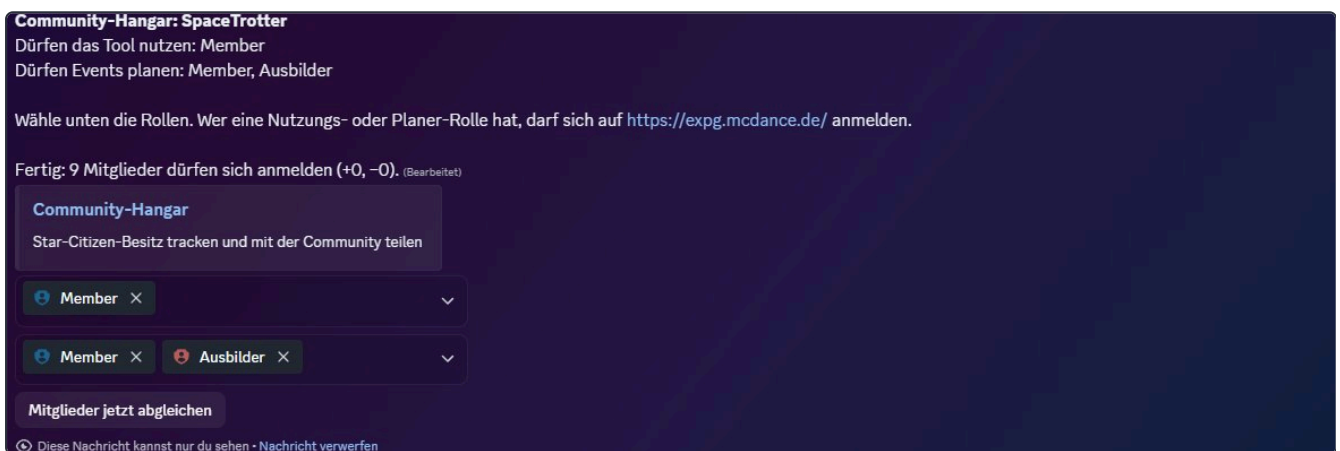


Die Einrichtungsnachricht mit zwei Rollenauswahlen und dem Knopf „Mitglieder jetzt abgleichen“.

- 1 „Wer darf das Tool nutzen?“: Wähle die Rollen deiner Mitglieder (bis zu 10). Rechts neben dem Rollennamen steht, wie viele Mitglieder sie hat. Wähle keine Bot-Rollen und keine Rollen von Personen, die noch keinen Zugang bekommen sollen.
- 2 „Wer darf Events planen?“: Optional die Planer-Rollen.
- 3 Die Auswahl gilt, sobald du das Menü schließt. Der Bot gleicht danach die Mitglieder ab.

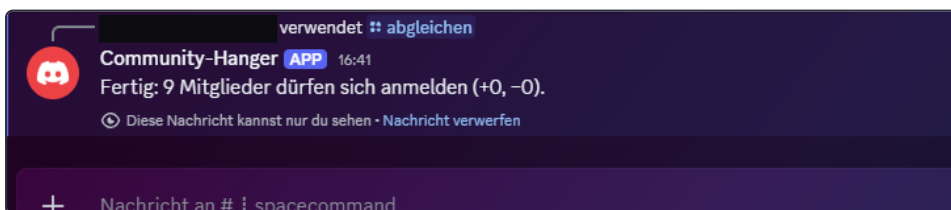


Rollenauswahl: Häkchen bei den Rollen, die das Tool nutzen dürfen (Namen auf dem Bild teilweise geschwärzt).



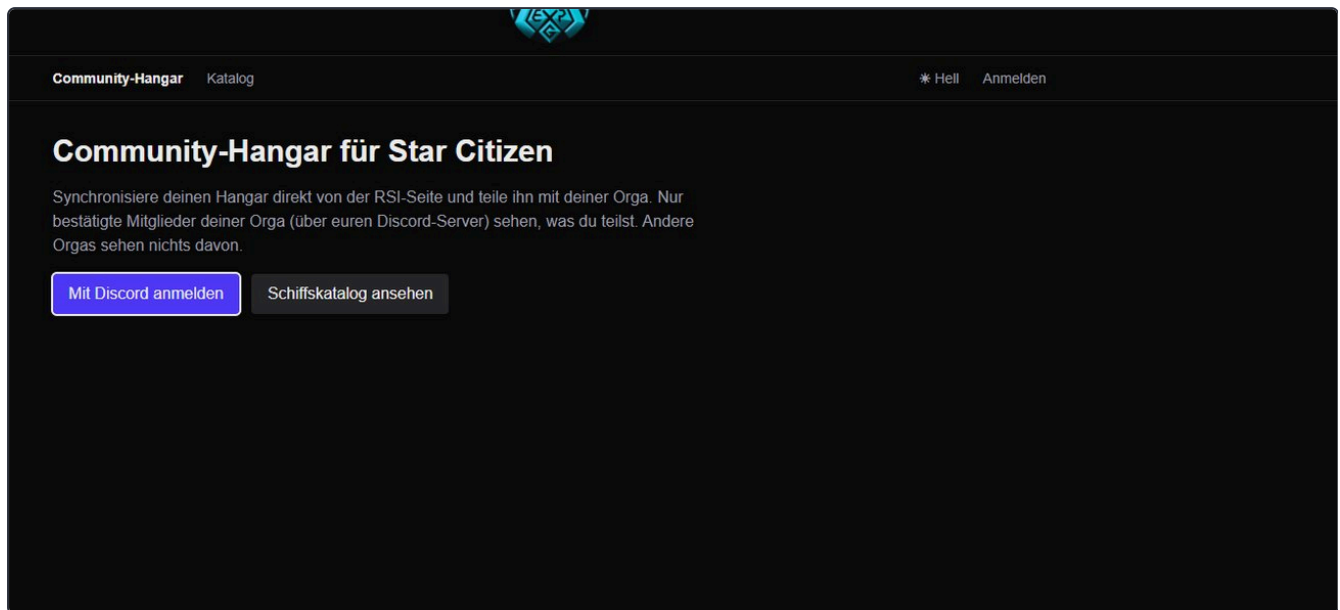
Fertig-Meldung: „9 Mitglieder dürfen sich anmelden“. Die Zahlen in Klammern sind die Änderungen seit dem letzten Abgleich. Nach der zweiten Auswahl steht dort „+0, -0“, weil schon beim ersten Mal abgeglichen wurde.

Der Befehl lässt sich beliebig oft wiederholen. Der Bot gleicht außerdem automatisch **einmal pro Stunde** ab. Wer die Rolle bekommt, darf sich anmelden. Wer sie verliert, darf es nicht mehr (laufende Sitzungen enden sofort, sein Konto wird gelöscht). Mit `/abgleichen` kann jede Person mit einer Nutzungs-Rolle (und jeder Server-Admin) sofort abgleichen.

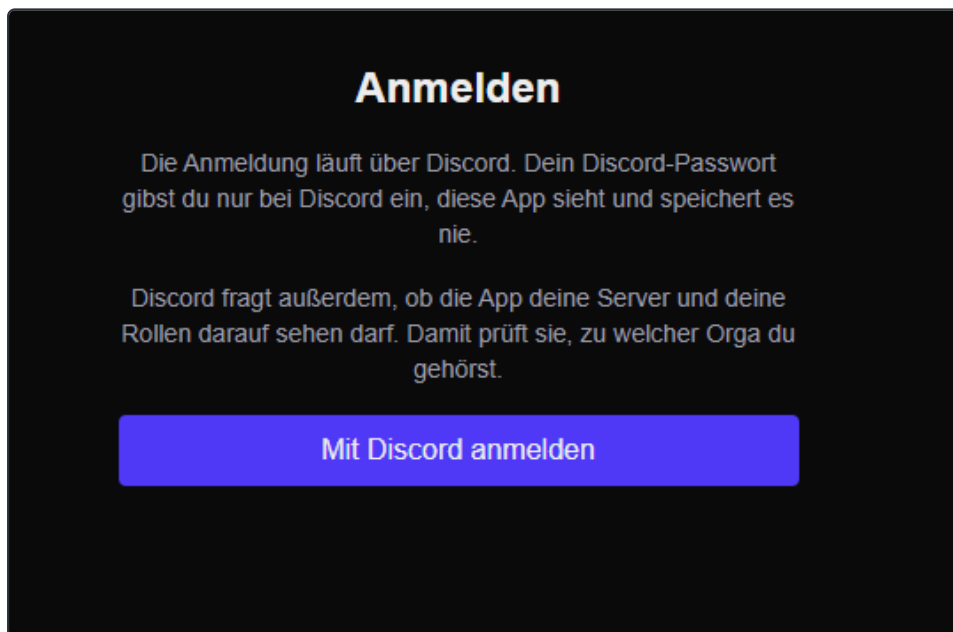


/abgleichen: die nur für dich sichtbare Antwort des Bots.

11 Anmelden

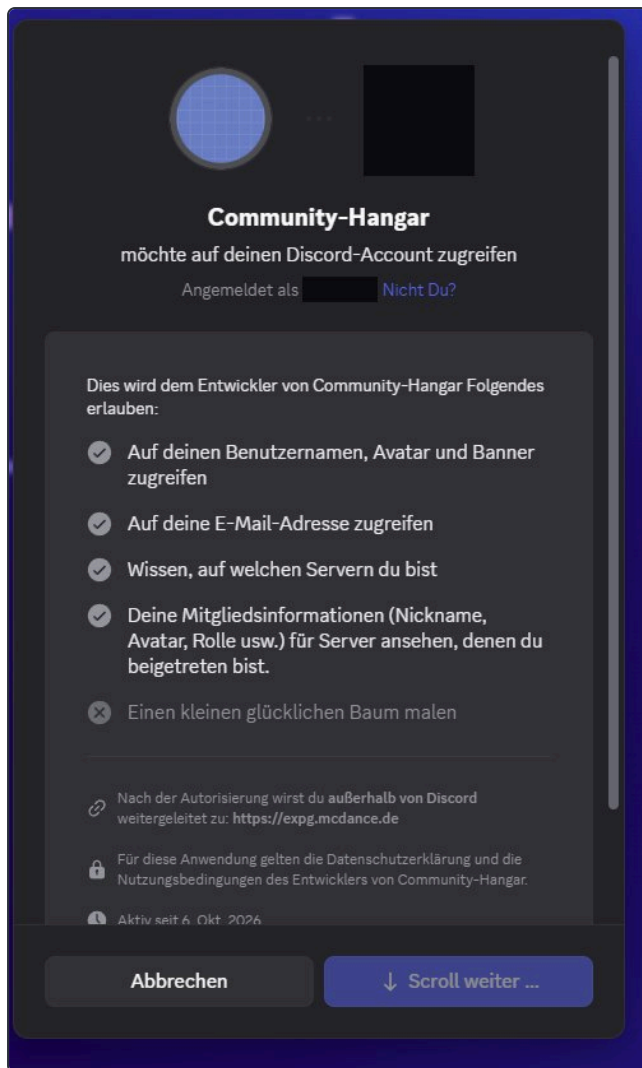


Die Startseite. Der Schiffskatalog ist auch ohne Anmeldung sichtbar.

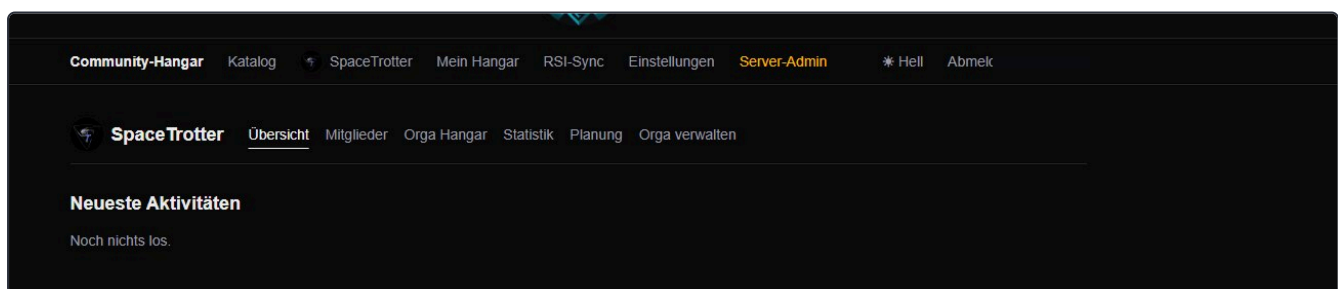


Auf „Mit Discord anmelden“ klicken. Dein Discord-Passwort gibst du nur bei Discord ein.

Discord zeigt eine Abfrage. Sie nennt drei Zugriffe: Benutzername, Avatar und Banner, auf welchen Servern du bist, und deine Mitgliedsinformationen (Nickname, Avatar, Rollen). Damit prüft die App, zu welcher Orga du gehörst. Scrolle nach unten, bis „Autorisieren“ aktiv wird, und bestätige selbst.



Discord-Abfrage bei der ersten Anmeldung. Unten steht, wohin du danach weitergeleitet wirst: deine Adresse. (Auf diesem Bild steht noch die Zeile zur E-Mail-Adresse. Die App fragt sie nicht mehr ab.)



Nach der Anmeldung siehst du deine Orga. „Server-Admin“ sehen nur Personen, die als SERVER_ADMIN_DISCORD_ID eingetragen sind.

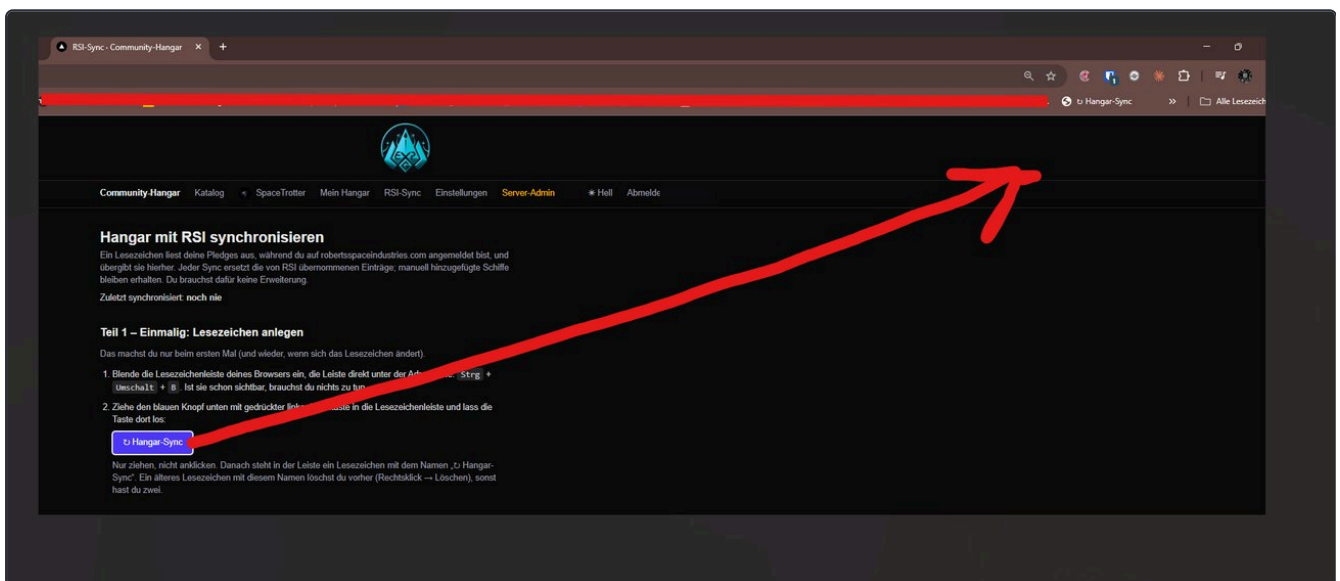
Ein Mitglied ohne Rolle sieht stattdessen: „Dieses Discord-Konto steht nicht auf der Zugangsliste.“

12 Ersten RSI-Sync machen

Jedes Mitglied überträgt seinen Hangar von der RSI-Seite. Öffne oben **RSI-Sync**.

Einmalig: Lesezeichen anlegen

- 1 Blende die Lesezeichenleiste deines Browsers ein: **Strg** + **Umschalt** + **B**.
- 2 Lösche vorher ein älteres Lesezeichen „**Hangar-Sync**“ (Rechtsklick → Löschen). Das Lesezeichen merkt sich die Adresse der App, von der du es gezogen hast; ein altes zeigt sonst auf die falsche.
- 3 Ziehe den blauen Knopf „**Hangar-Sync**“ mit gedrückter linker Maustaste in die Leiste. Nur ziehen, nicht anklicken.



Den Knopf in die Lesezeichenleiste ziehen.

Bei jedem Sync

- 1 Klicke auf der Seite RSI-Sync auf „**RSI-Pledge-Seite öffnen**“. Es öffnet sich ein **neuer Tab** mit RSI. Diese Seite bleibt offen und wartet.
- 2 Melde dich im neuen Tab bei RSI an, falls nötig. Du musst auf der Seite „**My Hangar**“ (Pledges) mit deiner Liste stehen. Der Tab-Titel zeigt das.
- 3 Klicke im **RSI-Tab** in der Lesezeichenleiste auf „**Hangar-Sync**“. Nicht im Tab der App, sondern im RSI-Tab.
- 4 Unten rechts läuft der Fortschritt („Lese Seite 4 ...“). Das dauert je nach Hangar ein paar Sekunden bis eine Minute, weil absichtlich langsam gelesen wird. Schließe den Tab nicht.
- 5 Danach schließt sich der RSI-Tab von selbst, und in der App erscheint eine **Vorschau**. Stimmt sie, klicke auf „**In meinen Hangar übernehmen**“. Mit „Abbrechen“ wird nichts verändert.

Community-Hangar

Katalog

SpaceTrotter

Mein Hangar

RSI-Sync

Einstellungen

Server-Admin

* Hell

Abmelde

Hangar mit RSI synchronisieren

Ein Lesezeichen liest deine Pledges aus, während du auf robertsspaceindustries.com angemeldet bist, und übergibt sie hierher. Jeder Sync ersetzt die von RSI übernommenen Einträge, manuell hinzugefügte Schiffe bleiben erhalten. Du brauchst dafür keine Erweiterung.

Zuletzt synchronisiert: noch nie

Teil 1 – Einmalig: Lesezeichen anlegen

Das machst du nur beim ersten Mal (und wieder, wenn sich das Lesezeichen ändert).

1. Blende die Lesezeichenleiste deines Browsers ein, die Leiste direkt unter der Adresszeile. `Strg + Umschalt + B`. Ist sie schon sichtbar, brauchst du nichts zu tun.
2. Ziehe den blauen Knopf unten mit gedrückter linker Maustaste in die Lesezeichenleiste und lass die Taste dort los:

Hangar-Sync

Nur ziehen, nicht anklicken. Danach steht in der Leiste ein Lesezeichen mit dem Namen „Hangar-Sync“. Ein älteres Lesezeichen mit diesem Namen löschst du vorher (Rechtsklick → Löschen), sonst hast du zwei.

Teil 2 – Bei jedem Sync

1. Wenn du auf `rsi.com` klickst, öffnet sich ein neuer Tab mit deinem RSI-Konto. Diese Seite hier bleibt offen und zeigt „Warte auf RSI“.

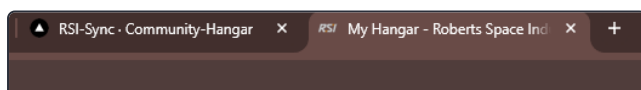
RSI-Pledge-Seite öffnen

2. Im neuen Tab: Melde dich bei RSI an, falls du es nicht schon bist. Du musst danach auf der Seite „My Hangar“ / „Pledges“ mit deiner Liste von Schiffen und Paketen stehen. Eine andere RSI-Seite (Pledge-Seite, Konto) reicht nicht.
3. Klicke im RSI-Tab in der Lesezeichenleiste auf „Hangar-Sync“. Nicht in diesem Tab, sondern im RSI-Tab.
4. Warte, bis das Lesezeichen fertig ist. Unten rechts auf der RSI-Seite läuft ein Kasten „Community-Hangar“ mit dem Fortschritt („Lese Seite 3 ...“). Das dauert je nach Hangar-Größe ein paar Sekunden bis eine Minute. Schließe den Tab in dieser Zeit nicht.
5. Danach schließt sich der RSI-Tab von selbst, und du siehst hier im Community-Hangar eine Vorschau mit allen erkannten Schiffen und Gegenständen. Wurde nichts erkannt oder fehlt etwas, brich mit „Abbrechen“ ab. Es wird nichts verändert.
6. Stimmt die Vorschau, klicke auf „In meinen Hangar übernehmen“. Erst jetzt wird dein Hangar aktualisiert, und die Seite lädt neu.

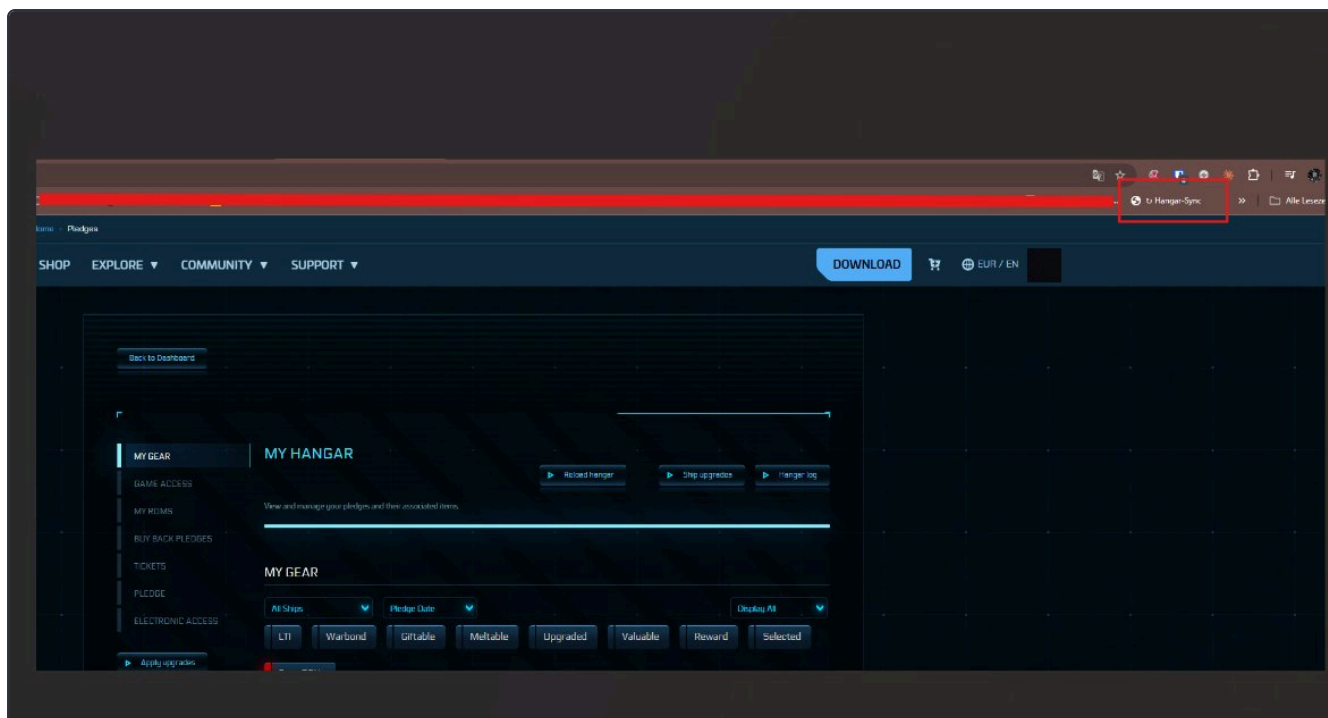
Wenn etwas nicht klappt

- Es öffnet sich kein neuer Tab: Der Browser hat das Öffnen blockiert. Klicke rechts in der Adresszeile auf das Pop-up-Symbol, erlaube Pop-ups für diese Seite und klicke den Knopf noch einmal.
- Beim Klick auf das Lesezeichen öffnet sich nur RSI: Du warst nicht auf der Pledge-Seite. Das Lesezeichen öffnet sie dann in einem neuen Tab für dich. Klicke es dort noch einmal an. Klickst du es in diesem Community-Hangar an, funktioniert es wie der Knopf oben, und die Vorschau erscheint hier.
- Im Kasten unten rechts steht ein Fehler: Meist bist du bei RSI nicht angemeldet. Melde dich an, lade die Pledge-Seite neu und klicke das Lesezeichen noch einmal an.
- Hier kommt nach dem Lesen keine Vorschau: Du hast diese Seite zwischendurch gewechselt oder geschlossen. Beginne mit Schritt 1 von vorn und lass diese Seite offen.

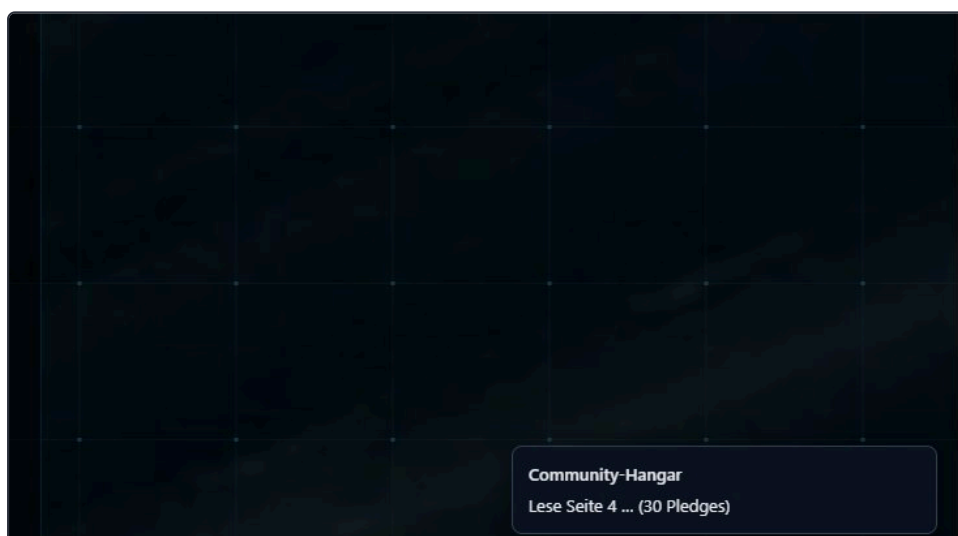
Auf „RSI-Pledge-Seite öffnen“ klicken.



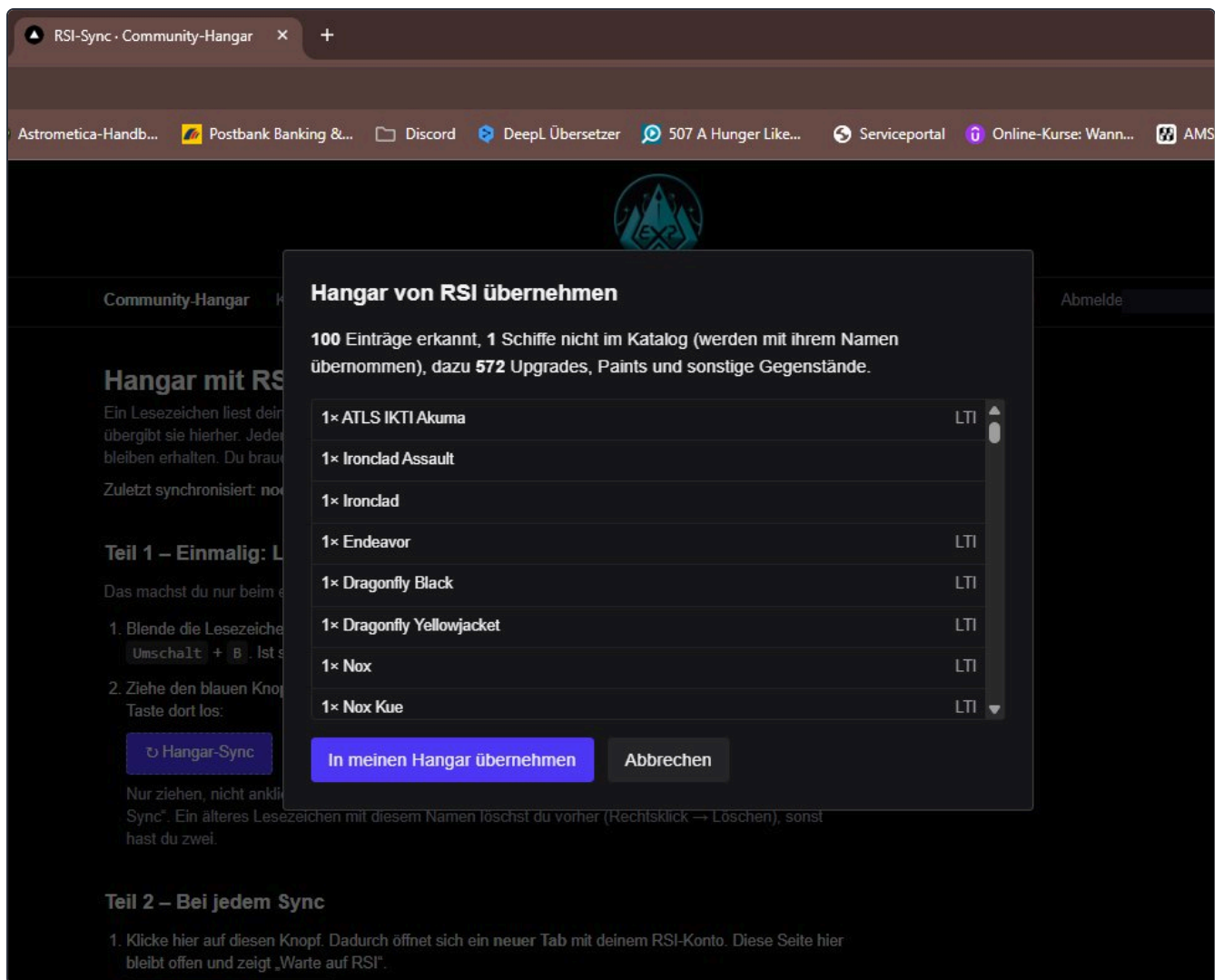
RSI öffnet sich in einem neuen Tab neben der App („My Hangar“).



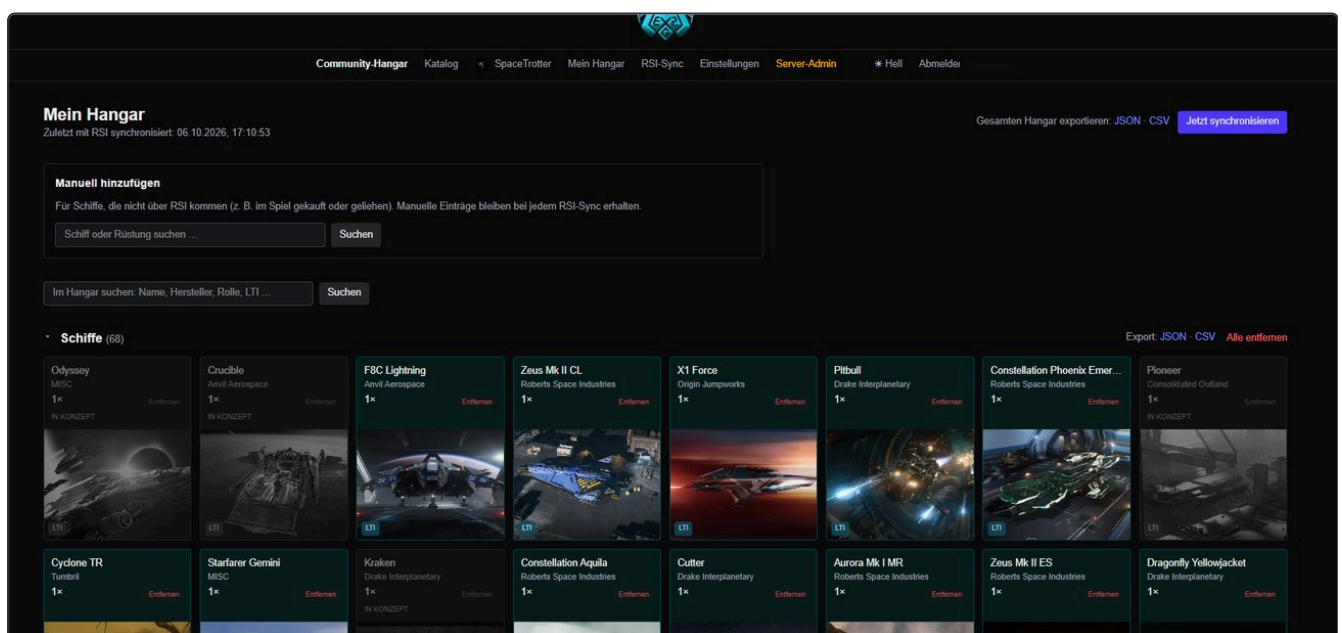
Im RSI-Tab auf der Seite „My Hangar“ das Lesezeichen „U Hangar-Sync“ anklicken.



Der Fortschritt unten rechts auf der RSI-Seite.



Die Vorschau in der App: erkannte Schiffe mit Stückzahl und LTI. „Schiffe nicht im Katalog“ werden mit ihrem Namen übernommen. Upgrades, Paints und Sonstiges werden nur gezählt.



Fertig: „Mein Hangar“ mit den Schiffen. Die Schiffsbilder lädt die App beim ersten Anzeigen von RSI, die ersten Aufrufe sind deshalb etwas langsamer.

13 Betrieb und Updates

Aufgabe	Befehl
Neue Version holen	<code>docker compose pull && docker compose up -d</code>
Logs ansehen	<code>docker compose logs -f app</code>
Katalog sofort abgleichen	<code>docker compose exec app php bin/catalog-sync.php</code>
Zugangslisten sofort abgleichen	<code>docker compose exec app php bin/sync-allowlist.php</code>
Schiffsbilder vorab laden	<code>docker compose exec app php bin/warm-images.php</code>
Datenbank sichern	<code>docker compose exec db sh -c 'mariadb-dump -u hangar -p"\$MARIADB_PASSWORD" hangar' > sicherung.sql</code>
Stoppen (Daten bleiben)	<code>docker compose down</code>
Alles löschen (Neuinstallation)	<code>docker compose down -v</code> löscht auch Datenbank und Bilder

Gesichert werden sollte die Datenbank. Die Bilder lädt die App bei Bedarf selbst nach.

Ohne eigenen Reverse-Proxy: Du kannst Caddy mitstarten, der das Zertifikat selbst holt. Dafür müssen die Ports 80 und 443 aus dem Internet erreichbar sein und die Domain auf den Server zeigen. Die dafür nötige Datei `docker-compose.override.yml` steht in `docs/schritt-fuer-schritt.md` im Repository (Abschnitt „Weg B“).

14 Wenn etwas nicht klappt

Symptom	Ursache und Lösung
Portal: „Private Anwendungen können keinen Standard-Autorisierungslink haben“	Unter Installation den Installations-Link auf Keine stellen (Kapitel 2.4), dann den Bot erneut privat speichern.
Container startet ständig neu („Restarting“)	<code>docker compose logs --tail 40 app</code> lesen. Die Meldung nennt den Grund, z. B. „Fehlende Einstellungen“ (Wert in der <code>.env</code> fehlt).
„Datenbank nicht erreichbar“ nach Passwortänderung	<code>DB_PASSWORD</code> wurde nach dem ersten Start geändert. Die Datenbank behält das erste Passwort: das alte wieder eintragen, oder bei leeren Daten <code>docker compose down -v</code> und neu starten.
<code>/healthz</code> zeigt „502 Bad Gateway“	Der Proxy erreicht die App nicht: Port, IP und Schema <code>http</code> im Proxy-Host prüfen, <code>docker compose ps</code> (läuft <code>app</code> ?).
Discord meldet „Invalid redirect_uri“	Die Redirect-Adresse im Portal muss genau <code>https://<DOMAIN>/api/auth/callback/discord</code> lauten und gespeichert sein.

Discord meldet „client_id ... snowflake“	<code>AUTH_DISCORD_ID</code> fehlt oder ist leer.
Portal lehnt die Endpunkt-URL ab	<code>DISCORD_PUBLIC_KEY</code> prüfen (Zeile ohne <code>#</code> ?), Seite von außen erreichbar? Nach <code>.env</code> -Änderung <code>docker compose up -d</code> .
<code>/einrichten</code> erscheint nicht	Befehle registrieren (Kapitel 8), Bot eingeladen (Kapitel 9)? Discord mit <code>Strg</code> + <code>R</code> neu laden, es kann ein paar Minuten dauern.
„Die Anwendung reagiert nicht“	Die Interaktions-Endpunkt-URL fehlt im Portal oder ist falsch (Kapitel 7). <code>docker compose logs app grep interactions</code> zeigt, ob Anfragen ankommen.
„Der Bot darf die Mitgliederliste nicht lesen“	Im Portal unter Bot den Server Members Intent einschalten und speichern.
„Dieser Server ist nicht freigeschaltet“	Die Server-ID in <code>ONBOARDING_GUILD_IDS</code> eintragen, dann <code>docker compose up -d</code> . Prüfe, dass du beim Einladen den richtigen Server gewählt hast.
Anmeldung: „nicht auf der Zugangsliste“	Die Person hat keine der gewählten Rollen, oder der Abgleich lief noch nicht (<code>/abgleichen</code>). Du selbst kommst über <code>SERVER_ADMIN_DISCORD_ID</code> immer herein.
Es öffnet sich kein neuer RSI-Tab	Der Browser hat das Öffnen blockiert. Pop-ups für die Seite erlauben (Symbol rechts in der Adresszeile) und den Knopf noch einmal klicken.
In der App kommt nach dem Lesen keine Vorschau	Du hast die App-Seite zwischendurch gewechselt oder geschlossen. Beginne mit Schritt 1 des Syncs von vorn und lass die App-Seite offen. Prüfe außerdem, dass du das Lesezeichen von deiner Instanz gezogen hast und kein altes auf eine andere Adresse zeigt.
Im Fortschrittskasten steht ein Fehler	Meist bist du bei RSI nicht angemeldet. Melde dich an, lade die Pledge-Seite neu und klicke das Lesezeichen noch einmal.
Katalog leer	<code>docker compose exec app php bin/catalog-sync.php</code> und die Ausgabe lesen (RSI-Server erreichbar?).
Schiffsbilder fehlen anfangs	Normal: Die App lädt sie beim ersten Anzeigen von RSI. Mit <code>bin/warm-images.php</code> vorab laden.